

DSS5201 Data Visualisation

Week 10 Data Visualisation with seaborn

David CHEW

Semester 2 AY25/26

Last Week: matplotlib

- **object-oriented approach** for `matplotlib`
- histogram (`ax.hist`) — distribution of a continuous variable
- bar plot (`ax.barh`) — compares categories
- scatter plot (`ax.scatter`) — relationship between two continuous variables
- line plot (`ax.plot`) — visualises trends over time

This Week: seaborn

- **Axes**-Level Functions
- **Figure**-Level Functions
- Customisation

Introduction

Introduction to seaborn

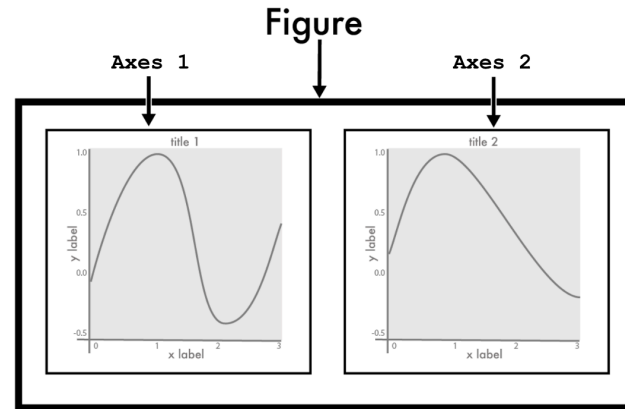
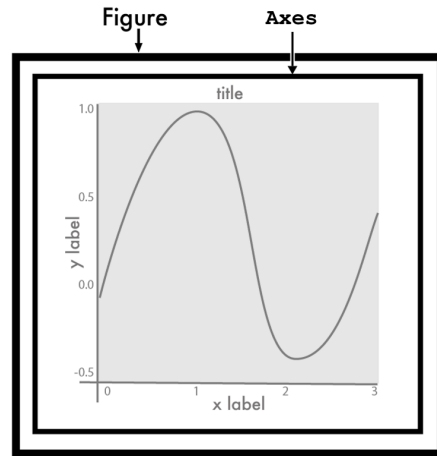
seaborn builds on matplotlib and works best with tidy data.

- By convention, we import it as `sns`.
- It comes with built-in themes. The default style is `darkgrid`.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 sns.set_theme(style = 'darkgrid')
```



Figure and Axes, again



`seaborn` functions can be classified into **axes** or **figure**-level.

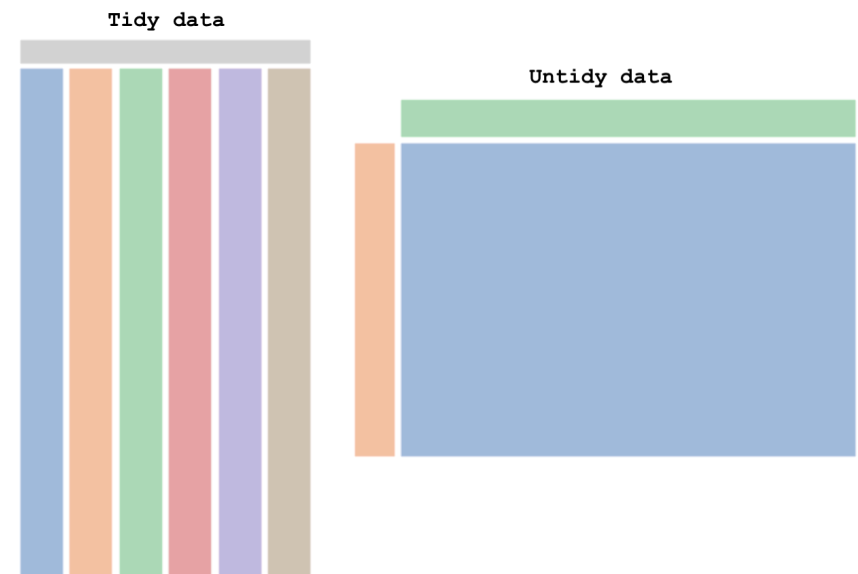
- An **axes-level** function works like `matplotlib` and returns a single plot on one **axes**.
- A **figure-level** function controls multiple **axes** within one **figure**.
 - These are the small multiples we came across previously.
 - Usually a `FacetGrid` object.

Data Structure for seaborn

`seaborn` works best with **tidy data**, where

- Each variable forms a column.
- Each observation forms a row.
- Each cell contains a single value.

Each variable can be assigned to a specific element in the plot, such as `x`, `y`, `hue`.



Axes-Level Functions

Axes-Level Functions

Axes-level function returns a single **axes** object.

- Drawn within an existing subplot.
- Can be customized using standard `matplotlib` commands.

We will look at three examples:

- Continuous distributions: `histplot()` and `kdeplot()`
- Pairwise relationships: `scatterplot()`

Penguins Data

- We will use the Penguins data on three species of penguins across three islands.
- The data structure is tidy.

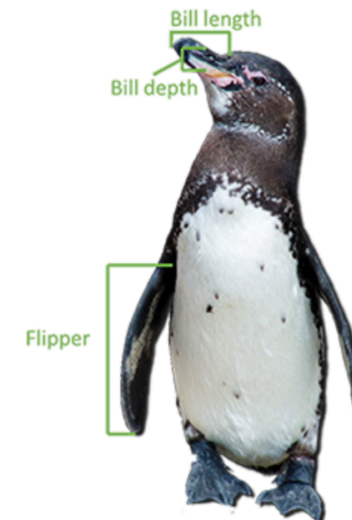
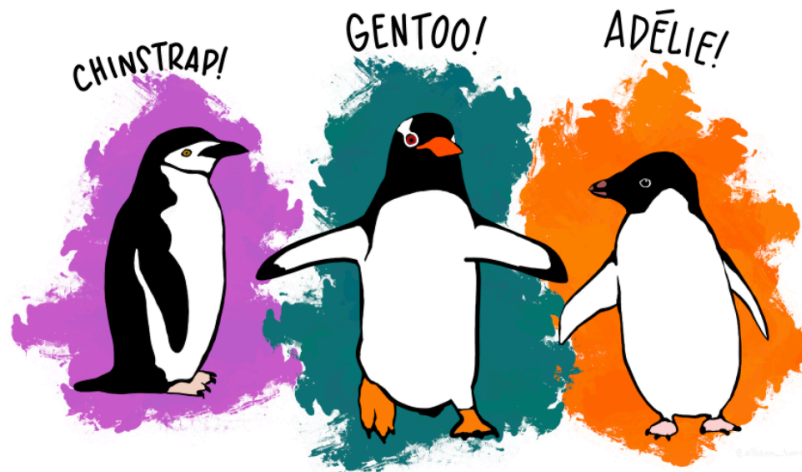
```
1 penguins = pd.read_csv('../data/penguins.csv').dropna(subset=['sex'])
2 penguins.head()
```



	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
0	Chinstrap	Dream	46.9	16.6	192	2700	female	2008
1	Adelie	Biscoe	36.5	16.6	181	2850	female	2008
2	Adelie	Biscoe	36.4	17.1	184	2850	female	2008
3	Adelie	Biscoe	34.5	18.1	187	2900	female	2008
4	Adelie	Dream	33.1	16.1	178	2900	female	2008

Penguins Data

- We will explore variations in penguin body mass across species, gender, and islands.
- How are these differences related to their physical characteristics, e.g., bill length, bill depth, and flipper length?

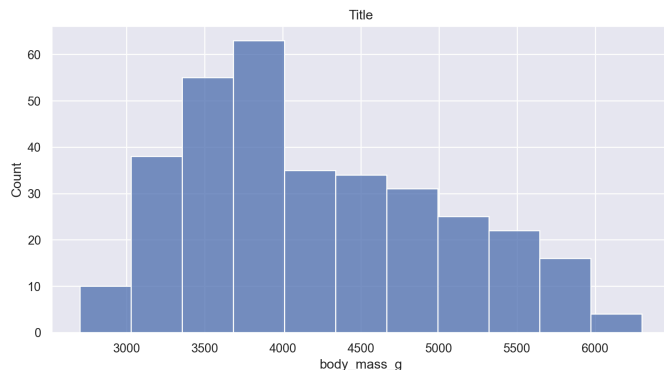


Source: Alison Horst

Simple vs Object-Oriented Approach

- We can use `seaborn` functions with or without the object-oriented approach.
- Here are two equivalent ways to create a simple histogram with a title.

```
1 # Simple
2
3 sns.histplot(
4     penguins,
5     x = 'body_mass_g')
6
7 plt.title('Title')
8 plt.show()
```



```
1 # Object-oriented
2 fig, ax = plt.subplots()
3 sns.histplot(
4     penguins,
5     x = 'body_mass_g',
6     ax = ax)
7 ax.set(title = 'Title')
8 plt.show()
```

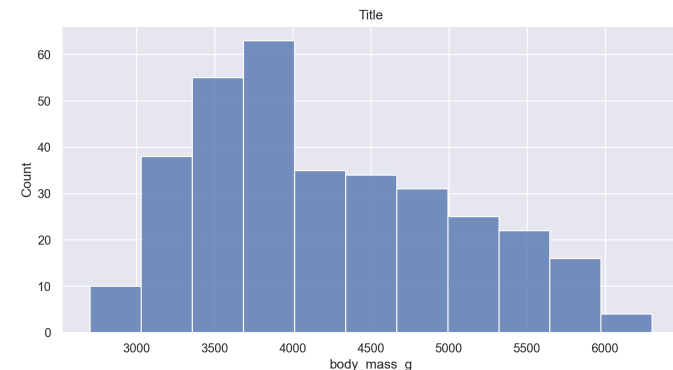
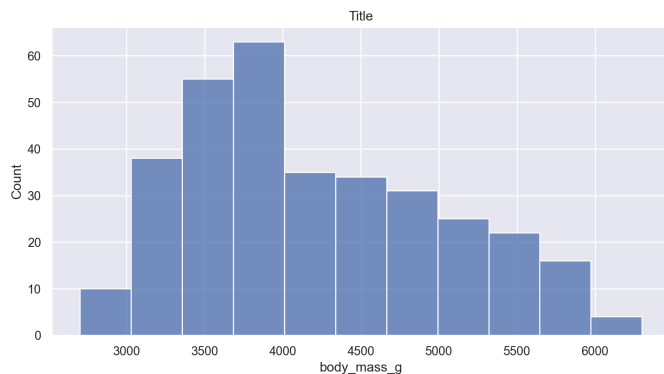


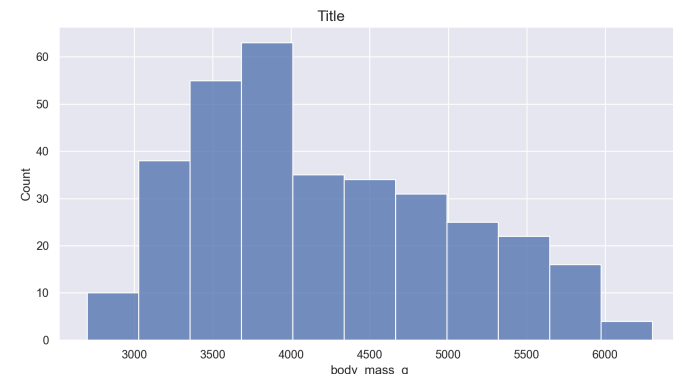
Figure-Level Object-Oriented Approach

- `seaborn` also supports **figure**-level object-oriented style.
- In this case, the **figure** and **axes** objects are created internally.

```
1 # Object-oriented
2 fig, ax = plt.subplots()
3 sns.histplot(
4     penguins,
5     x = 'body_mass_g',
6     ax = ax)
7 ax.set(title = 'Title')
8
9 plt.show()
```



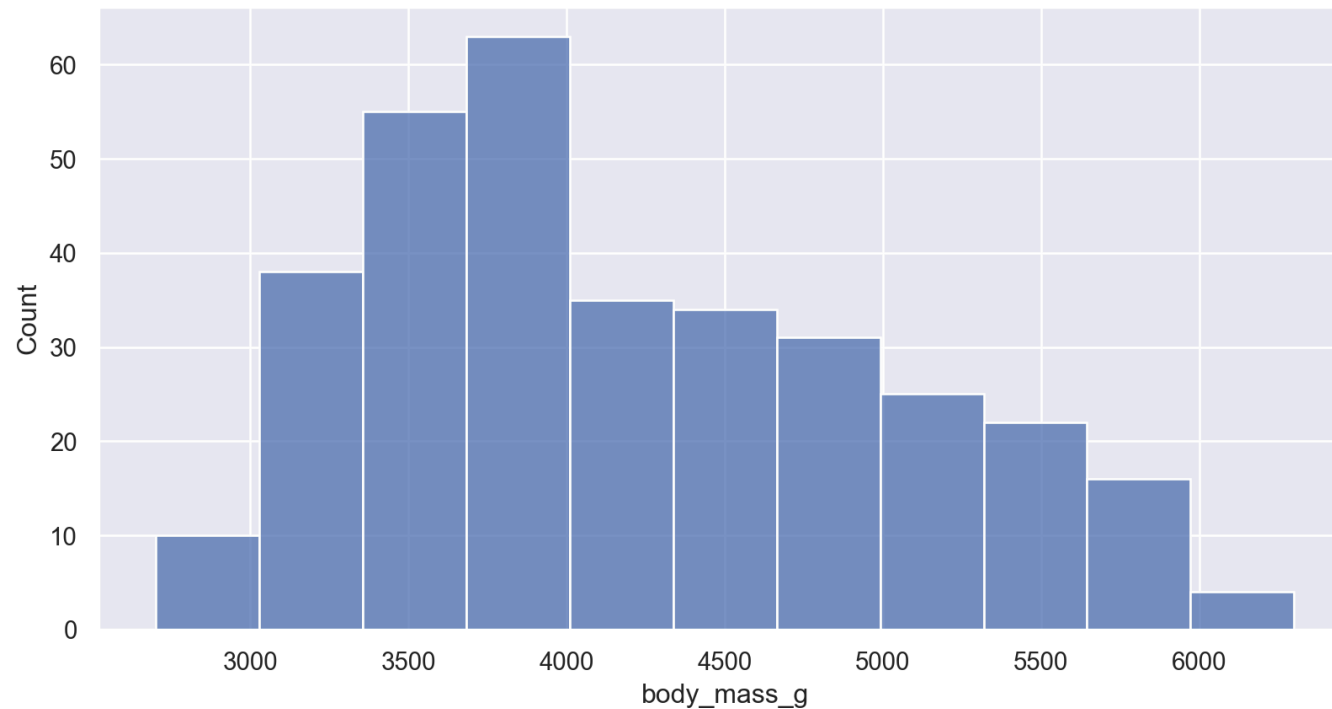
```
1 # Figure-level object-oriented
2
3 g = sns.displot(
4     penguins,
5     x = 'body_mass_g',
6     kind = 'hist')
7 g.fig.suptitle('Title', y=1.01)
8 g.fig.set_size_inches(10, 5)
9 plt.show()
```



Histogram

- How is the body mass distributed among this sample of penguins?
- To visualise the distribution of a continuous variable, we typically use a **histogram**.

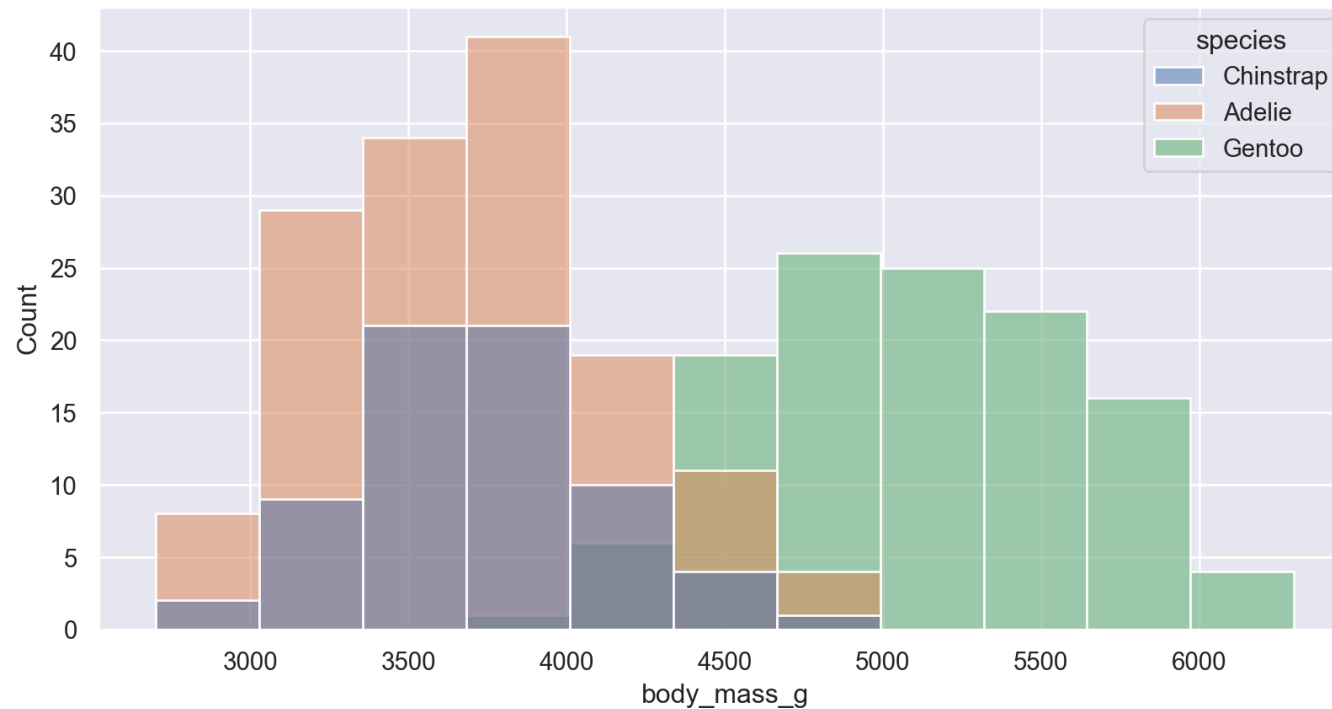
```
1 sns.histplot(penguins, x = 'body_mass_g')  
2 plt.show()
```



Histogram: Overlaid

- It is often helpful to map additional variables to the visual elements of the plot.
- By default, `seaborn` overlays the histograms for different categories.

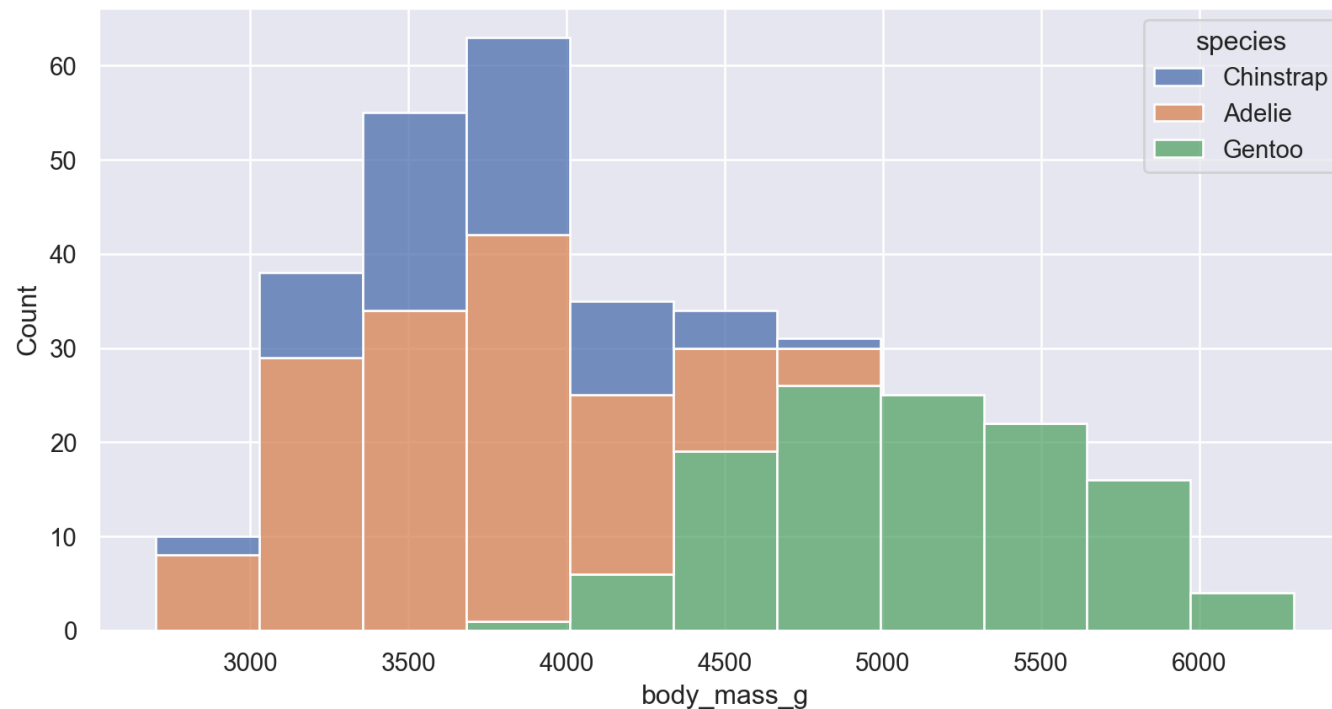
```
1 sns.histplot(penguins, x = 'body_mass_g', hue = 'species')  
2 plt.show()
```



Histogram: Stacked

- A **stacked histogram** emphasizes the part-to-whole relationship between categories.
- Compares both the shape and the composition of the data.

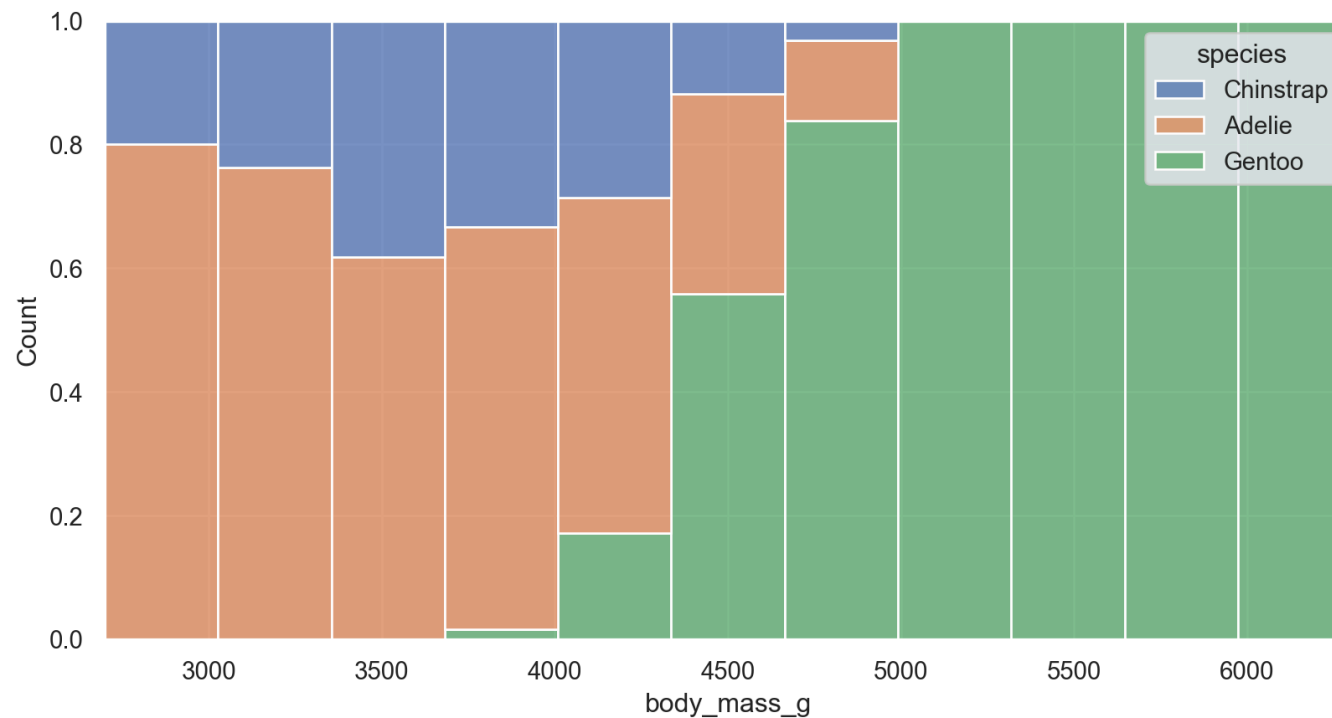
```
1 sns.histplot(penguins, x = 'body_mass_g', hue = 'species', multiple='stack')  
2 plt.show()
```



Histogram: Proportion Stacked

- ... shows the relative proportions of each category within the bin.
- The height of each bar equals to 1 (100%).

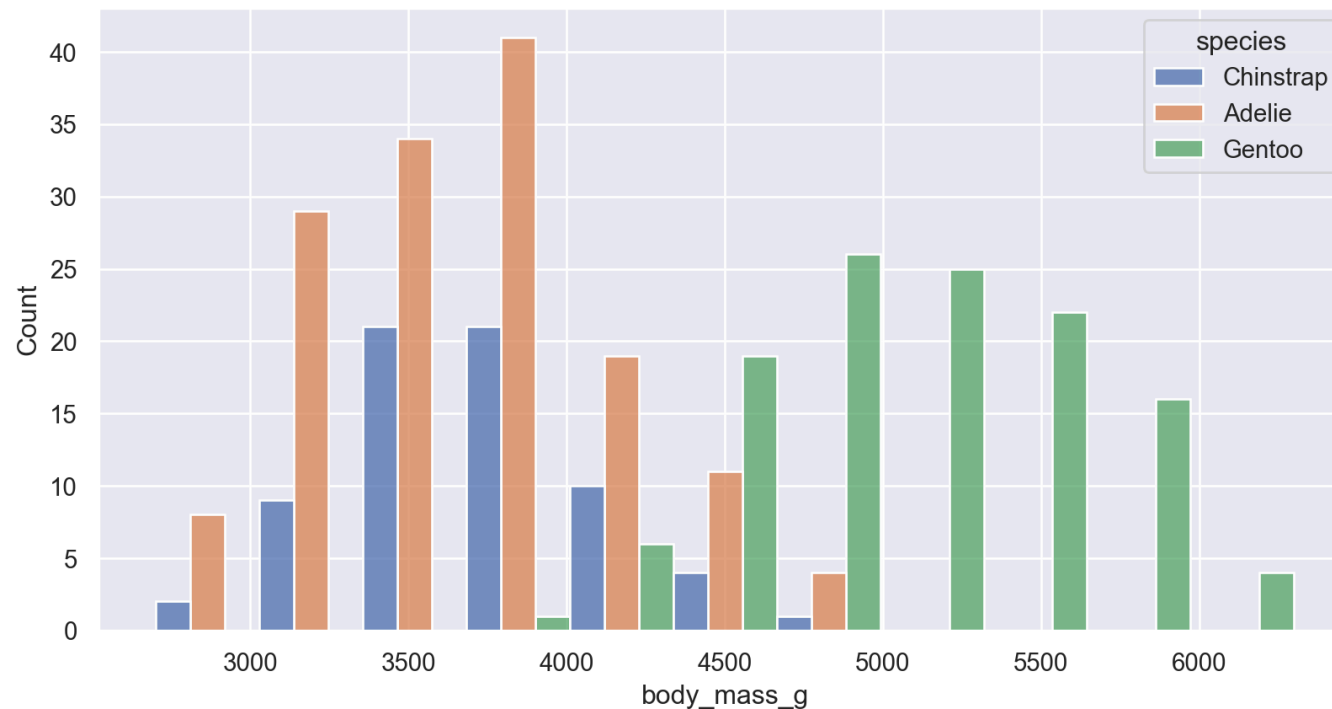
```
1 sns.histplot(penguins, x = 'body_mass_g', hue = 'species', multiple='fill')  
2 plt.show()
```



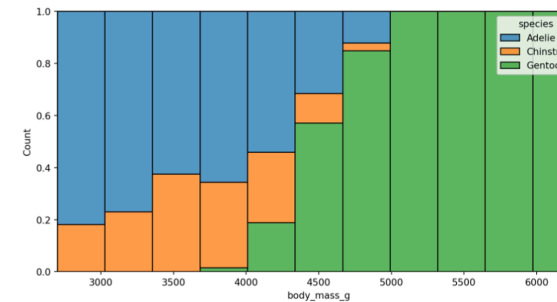
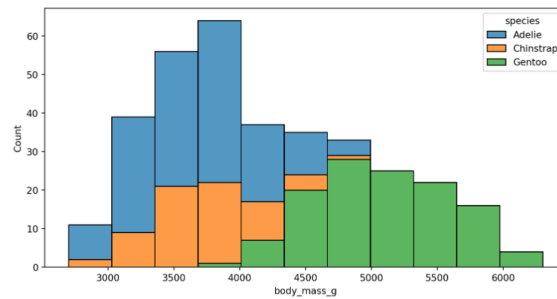
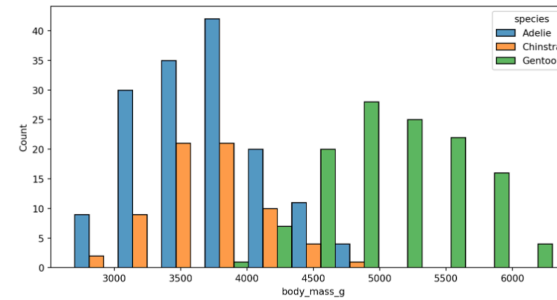
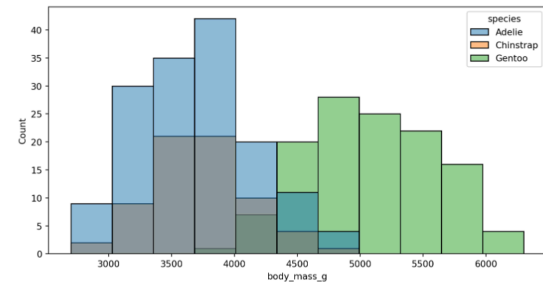
Histogram: Dodged

- A **dodged histogram** places groups side by side.
- It avoids overlap and make the differences between categories easier to see.

```
1 sns.histplot(penguins, x = 'body_mass_g', hue = 'species', multiple='dodge')  
2 plt.show()
```



Histogram: Different Types

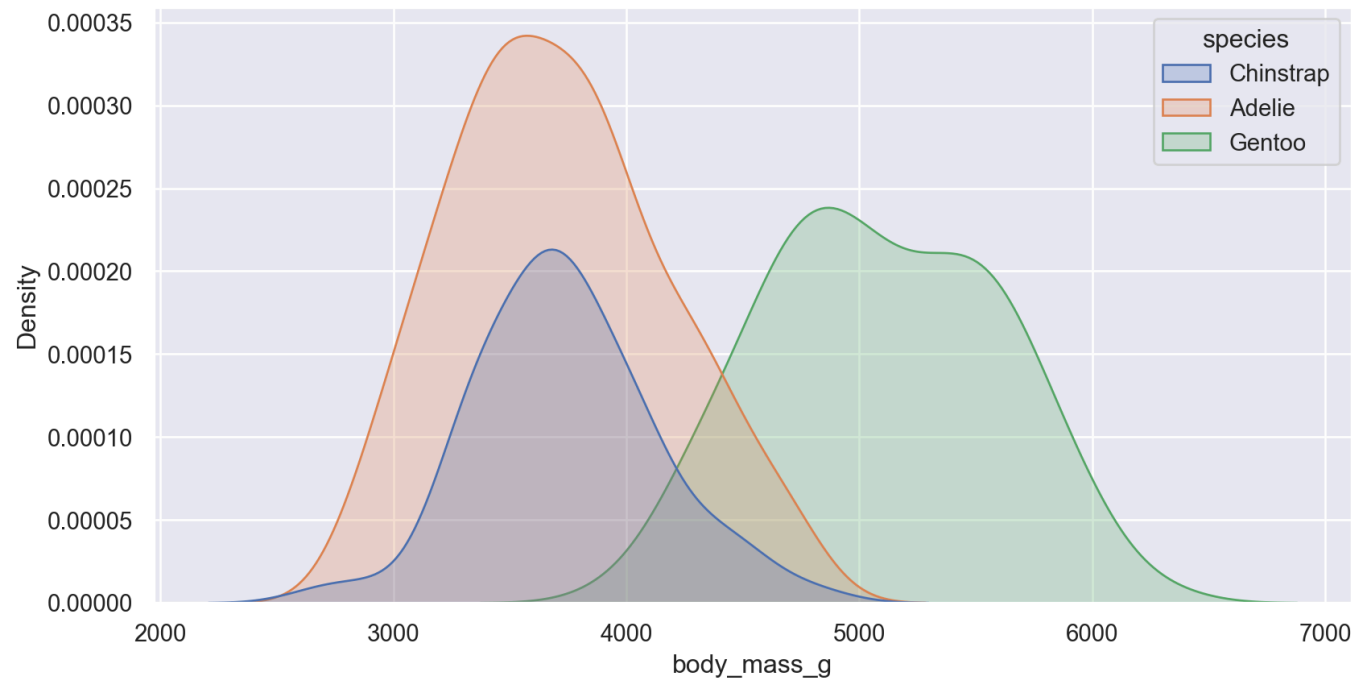


Type	Use case
Default (overlay)	Compare overall shapes across groups. Emphasize the overall pattern (but KDE often works better).
Dodged	Compare distributions directly between groups. Emphasize side-by-side comparison.
(Proportion) stacked	Show how subgroups contribute to the total. Emphasize (relative) composition by group.

Kernel Density Estimation (KDE) Plot

- Consider the **kernel density estimation plot**. It estimates the density using `scipy` internally.
- The smooth curves makes it easier to compare the overall patterns.

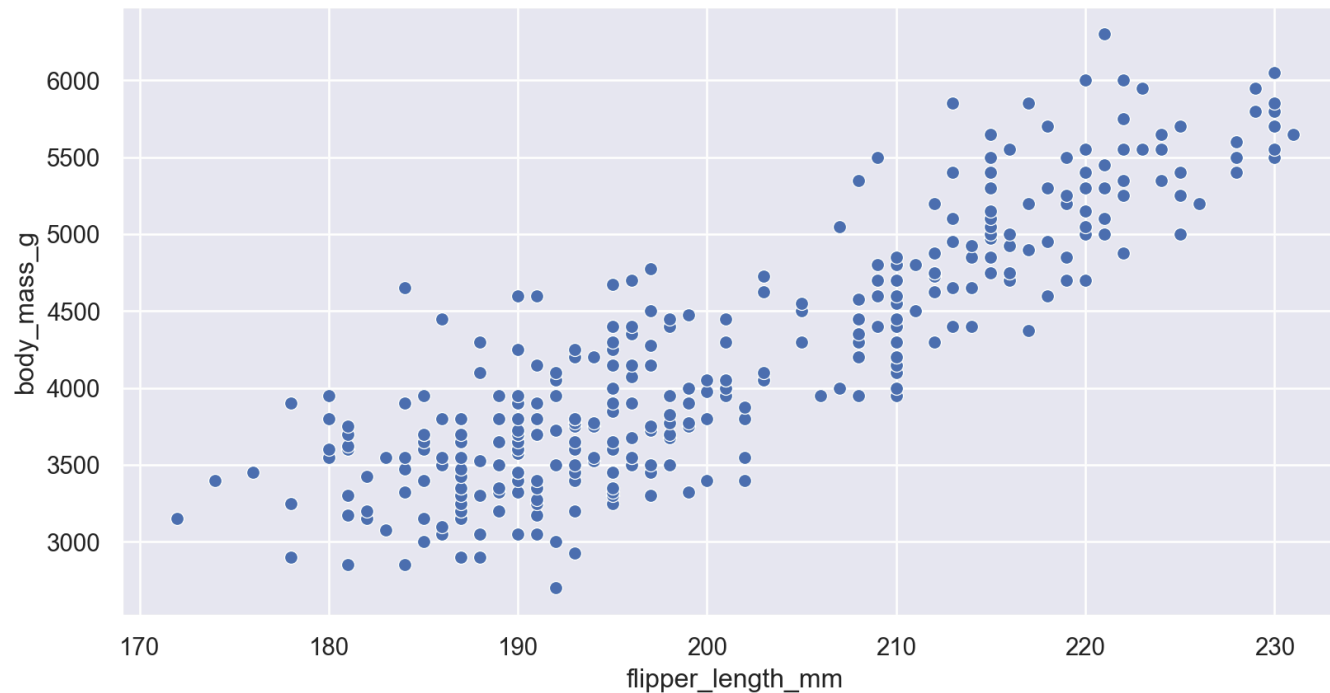
```
1 sns.kdeplot(penguins, x = 'body_mass_g', hue = 'species', fill = True)  
2 plt.show()
```



Scatter Plot

- Do penguins with longer flippers tend to be heavier?
- The **scatter plot** visualises relationships between continuous variables.

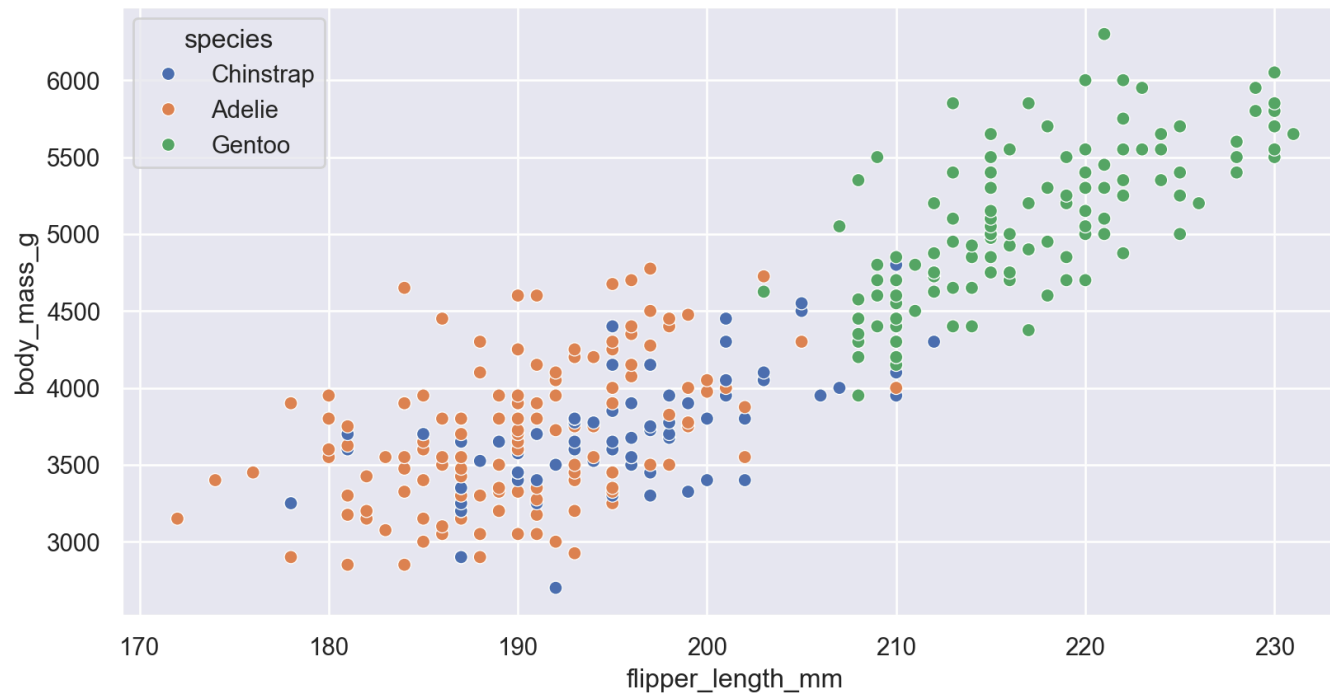
```
1 sns.scatterplot(penguins,  
2                 x = 'flipper_length_mm', y = 'body_mass_g')  
3 plt.show()
```



Scatter Plot

- We can add a third variable with `hue` to distinguish groups.
- It allows us to look at how variables are related for different subgroups.

```
1 sns.scatterplot(penguins,  
2                 x = 'flipper_length_mm', y = 'body_mass_g', hue = 'species')  
3 plt.show()
```



Scatter Plot

seaborn vs matplotlib

- `seaborn` automatically handles grouping, color mappings, and legends.
- It is often much cleaner to write, especially for multivariate visualisations.

```
1 sns.scatterplot(penguins,  
2                 x = 'flipper_length_mm', y = 'body_mass_g', hue = 'species')  
3 plt.show()
```

```
1 fig, ax = plt.subplots()  
2 for x in penguins['species'].unique():  
3     subset = penguins[penguins['species'] == x]  
4     ax.scatter(subset['flipper_length_mm'],  
5               subset['body_mass_g'],  
6               label = x)  
7 ax.legend(title = 'Species', loc = 'upper left')  
8 plt.show()
```

Scatter Plot

Legend, Title, and Axes Labels

- We can add title and labels to the plot.
- `plt.legend()` can be used to adjust the legend.

```
1 sns.scatterplot(penguins,  
2                 x = 'body_mass_g', y = 'flipper_length_mm', hue = 'species')  
3 plt.xlabel('Body mass (g)')  
4 plt.ylabel('Flipper length (mm)')  
5 plt.title('Body mass and flipper length')  
6 plt.legend(title = 'Species', loc = 'upper left')  
7 plt.show()
```



Scatter Plot

Legend, Title, and Axes Labels

The final scatter plot with proper axis labels, legends, and a title:



Figure-Level Functions

Figure-Level Functions



We now move to `seaborn`'s **figure-level** approach.

- Useful for faceted or multi-panel visualisations for comparing subgroups.
- The idea of **small multiples** from the visualisation principles lecture.

Figure-Level Functions

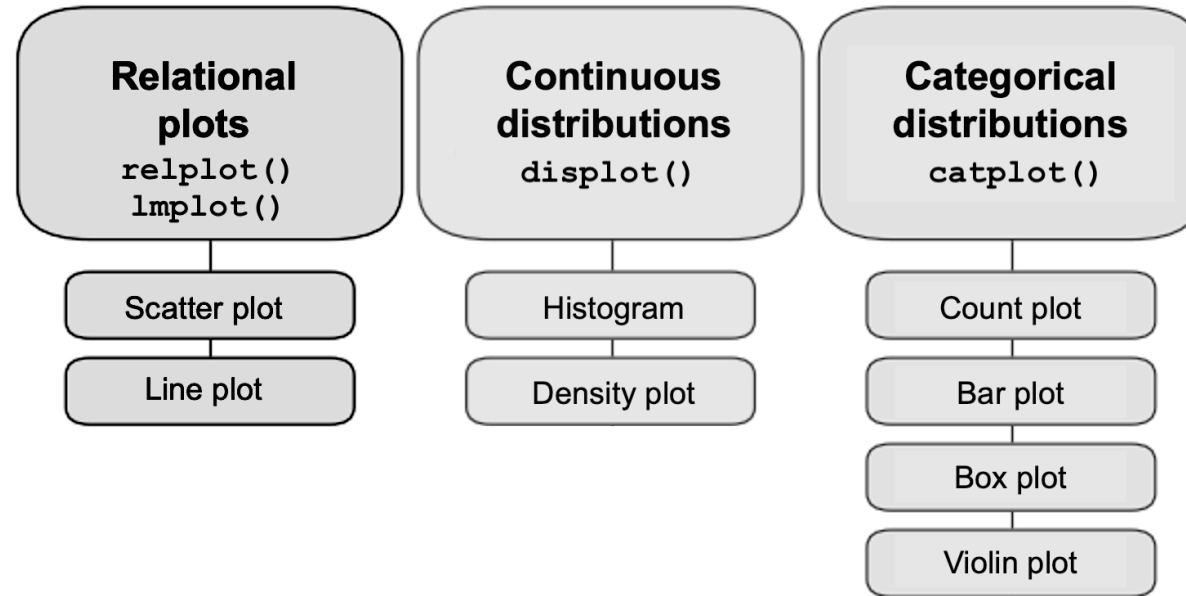


Figure-level functions create faceted visualisations.

- ... automatically manage the overall **figure**, shared axes, and legends.
- ... return `FacetGrid` objects that control entire layouts.

Relational Plots: `relplot()`

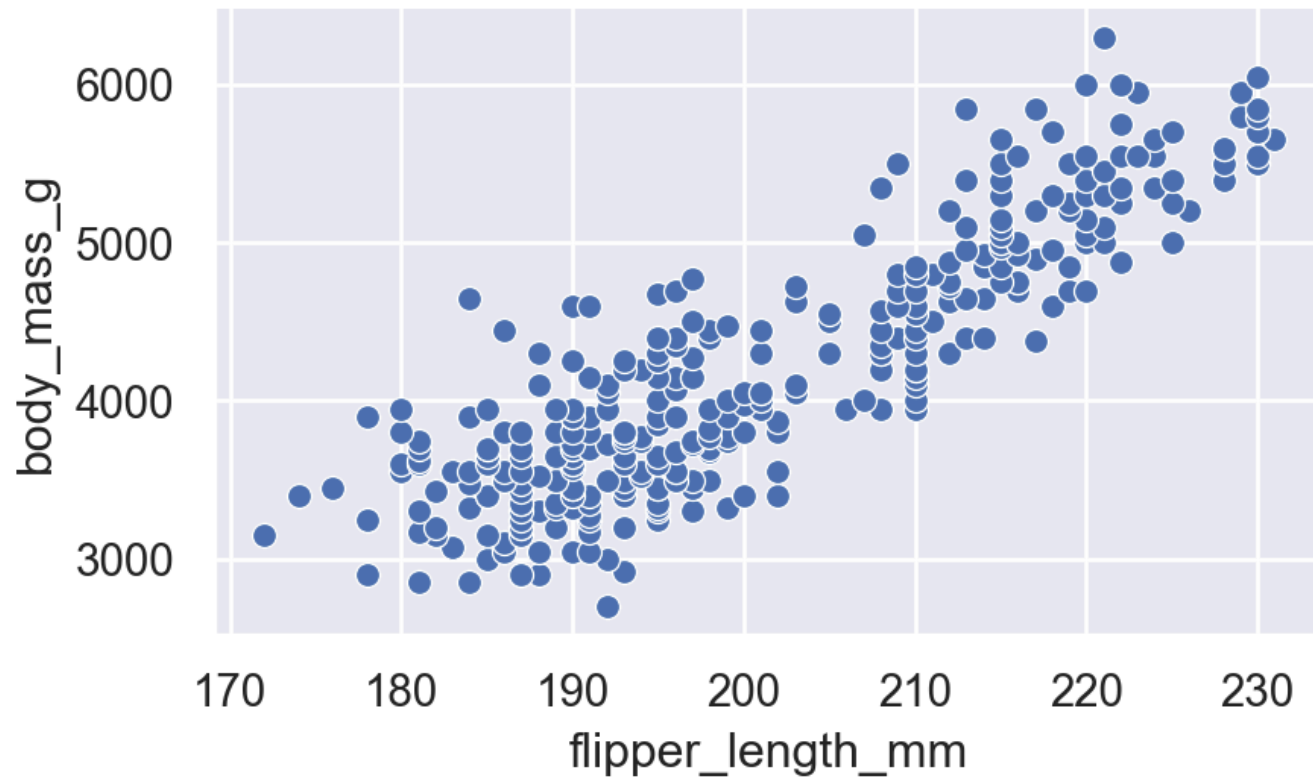
`relplot()` visualises statistical relationships between continuous variables.

- The `kind` argument selects the type of plot.
 - `kind = 'scatter'`: Scatter Plot (the default).
 - `kind = 'line'`: Line plot.
- The relationship can be shown for subsets of data with faceted plots.
- Especially useful when we want a multi-panel layout.

Relational Plots: relplot()

Single Scatter Plot

```
1 sns.relplot(penguins, kind = 'scatter',  
2             x = 'flipper_length_mm', y = 'body_mass_g',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

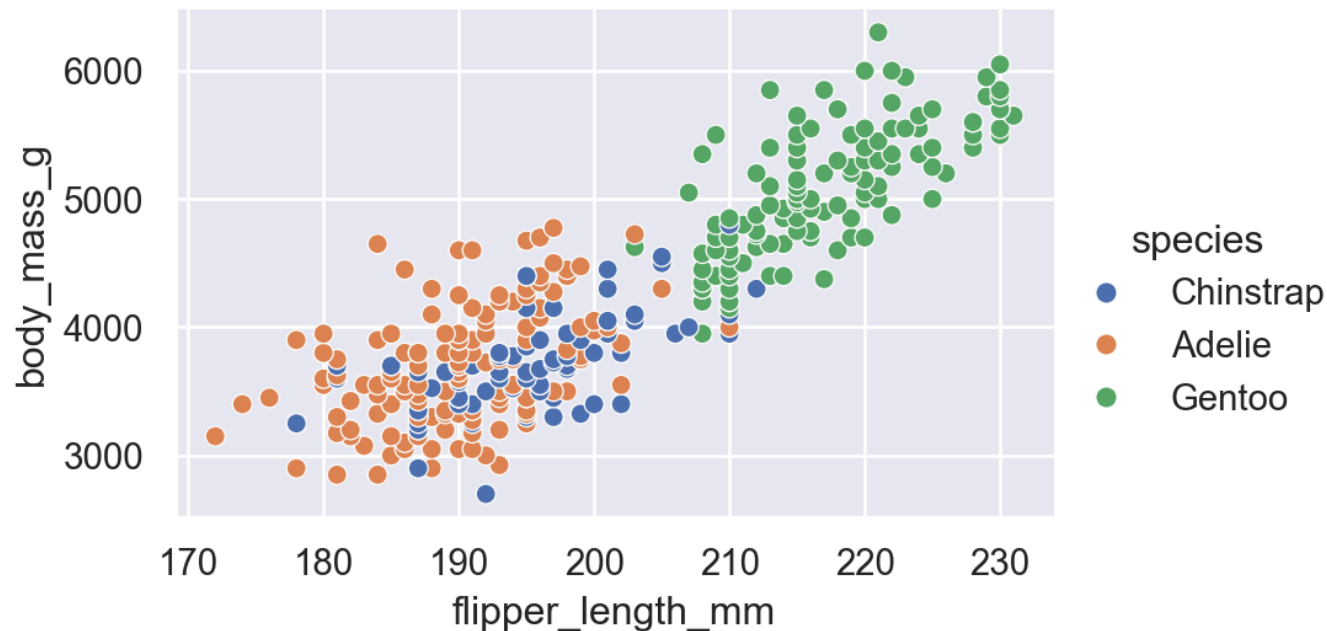


Relational Plots: `relplot()`

Single Scatter Plot: Mapping Additional Variable(s)

- Having additional variables can show multi-dimensional relationships

```
1 sns.relplot(penguins, kind = 'scatter', hue = 'species',  
2             x = 'flipper_length_mm', y = 'body_mass_g',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

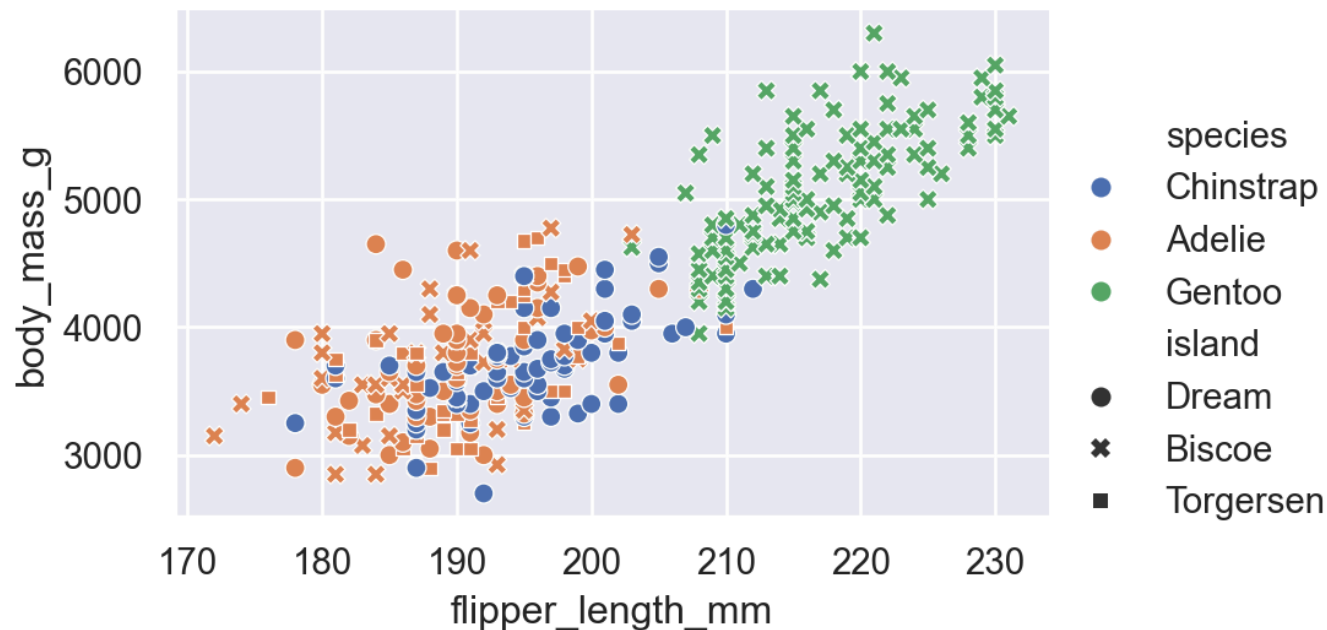


Relational Plots: relplot()

Single Scatter Plot: Mapping Additional Variable(s)

- ... but avoid making an **overly-complicated plot!!**

```
1 sns.relplot(penguins, kind = 'scatter', hue = 'species', style = 'island',
2             x = 'flipper_length_mm', y = 'body_mass_g',
3             height = 3, aspect = 1.618)
4 plt.show()
```

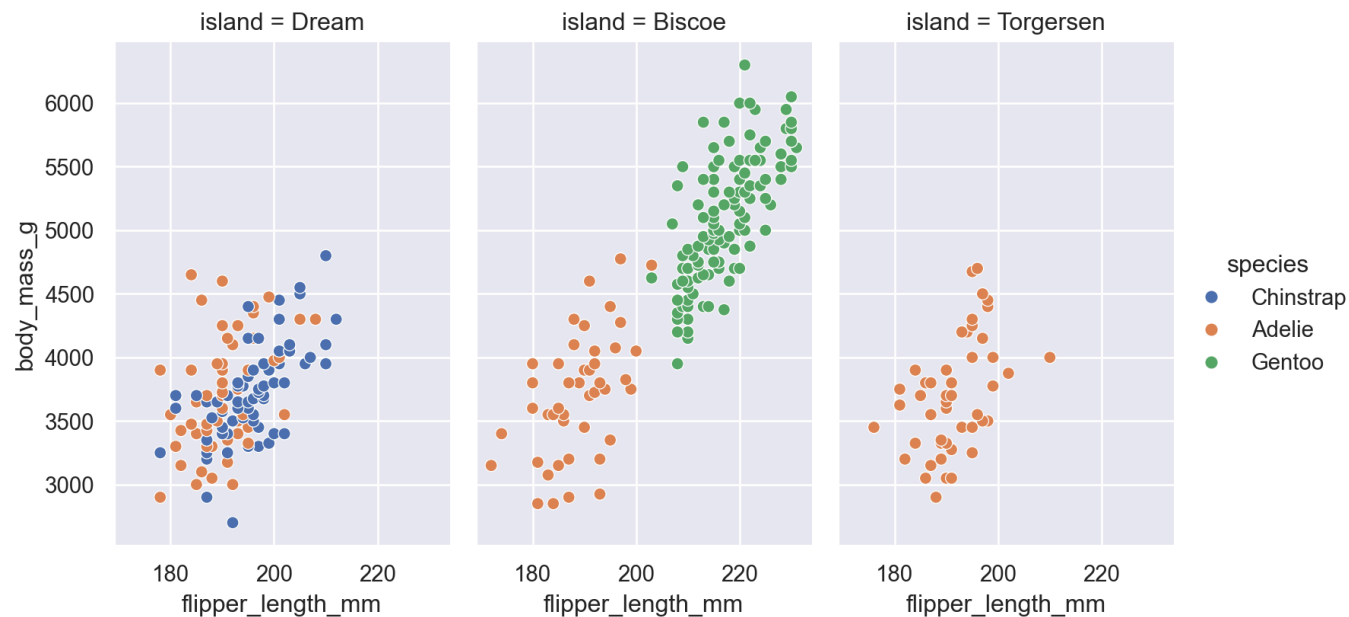


Relational Plots: relplot()

Faceted Plots

- We can assign a variable across columns (`col`) to create a **faceted plot**.

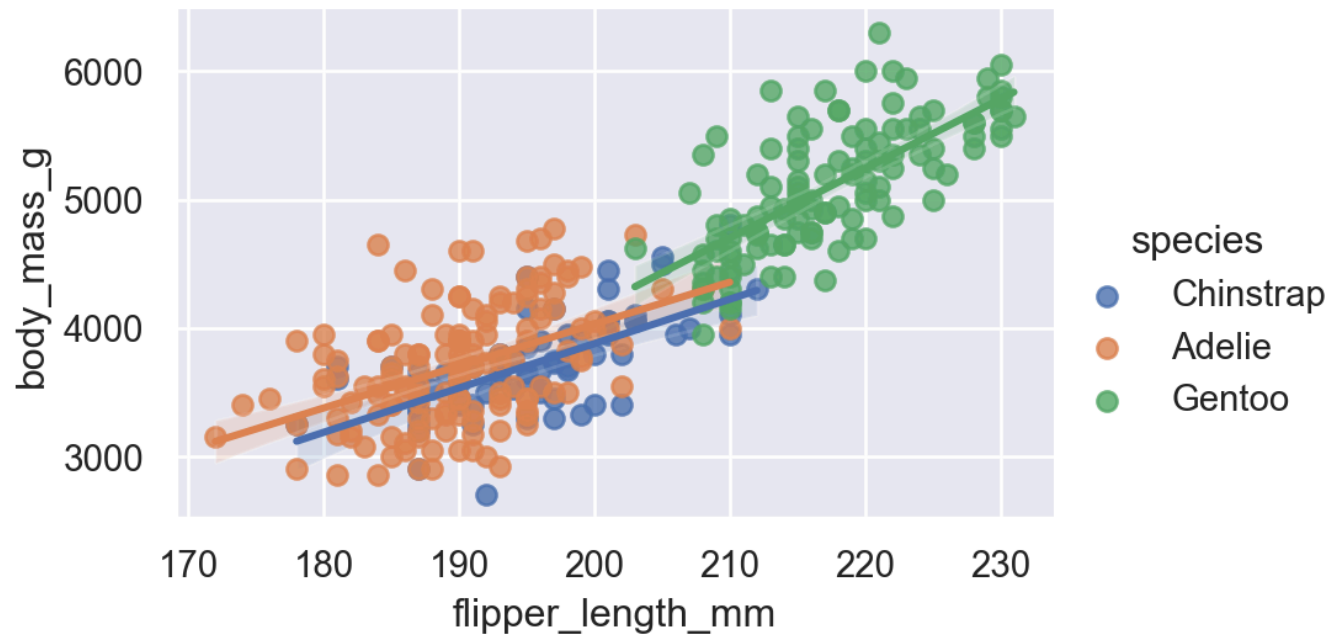
```
1 sns.relplot(penguins, kind = 'scatter', hue = 'species', col = 'island',  
2             x = 'flipper_length_mm', y = 'body_mass_g',  
3             height = 4.5, aspect = 0.618)  
4 plt.show()
```



Trend Lines: `lmpplot()`

- We can use `lmpplot()` to visualise relationships with a linear regression fit.

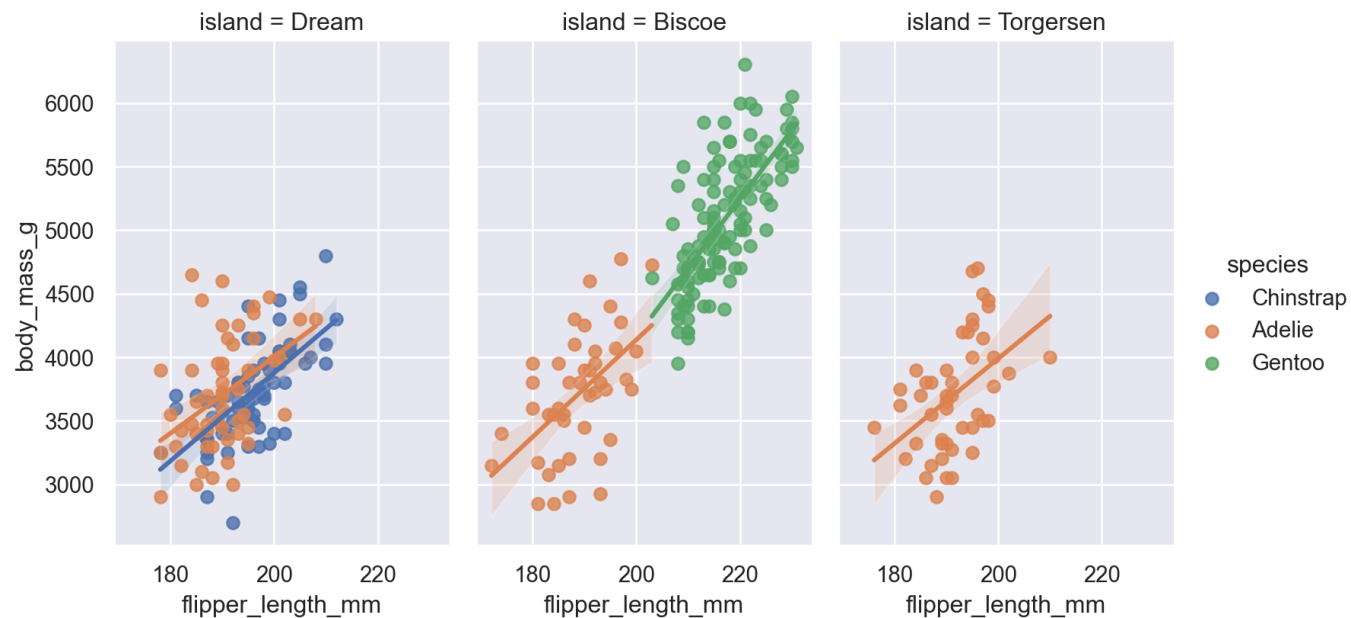
```
1 sns.lmpplot(penguins, hue = 'species',  
2             x = 'flipper_length_mm', y = 'body_mass_g',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```



Trend Lines: `lmplot()`

- `lmplot()` also supports faceting to explore relationships within subgroups.

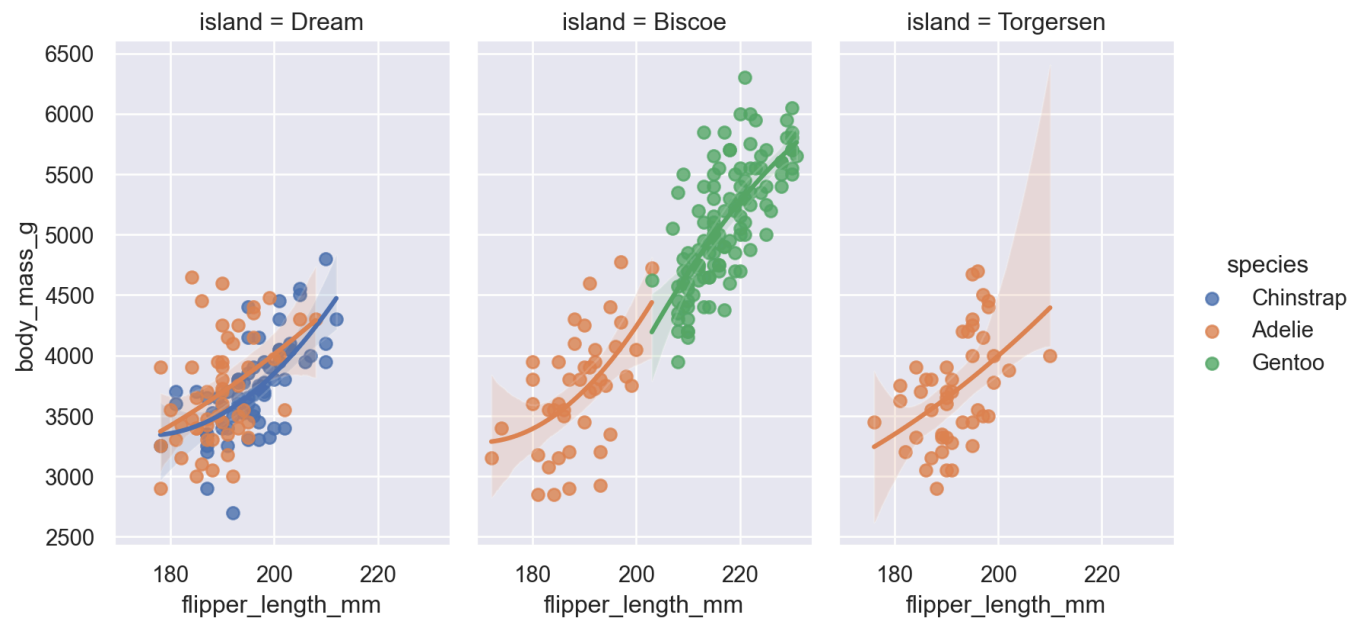
```
1 sns.lmplot(penguins, hue = 'species', col = 'island',
2           x = 'flipper_length_mm', y = 'body_mass_g',
3           height = 4.5, aspect = 0.618)
4 plt.show()
```



Trend Lines: `lmplot()`

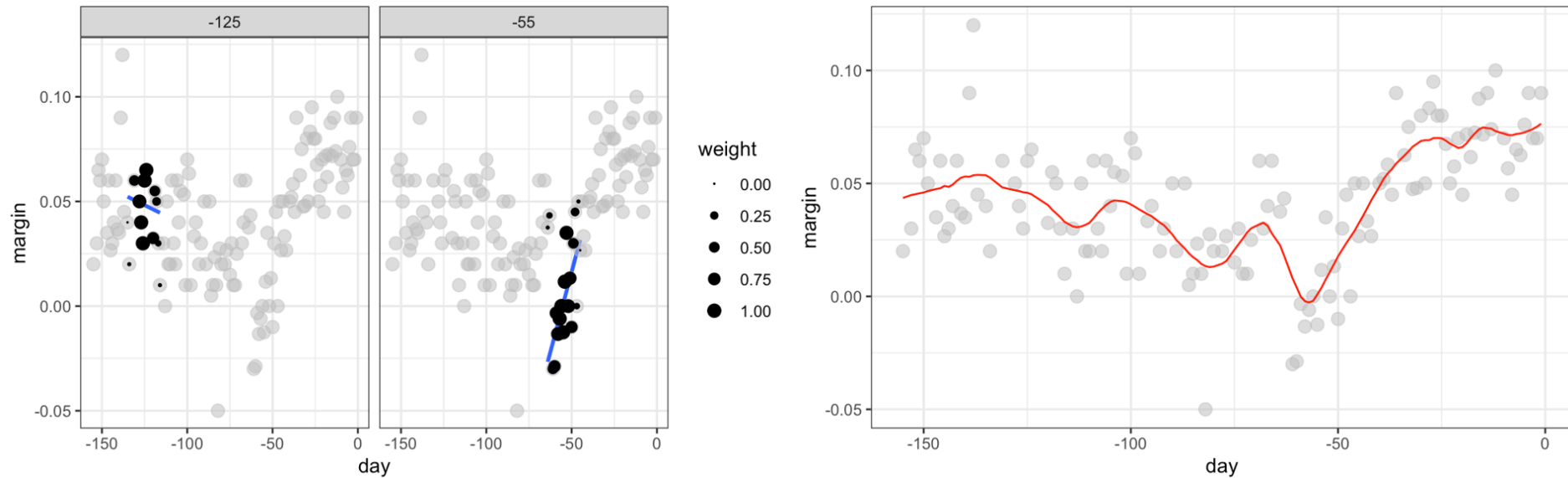
- `order = n` makes the model a n -th degree polynomial.

```
1 sns.lmplot(penguins, hue = 'species', col = 'island',
2           x = 'flipper_length_mm', y = 'body_mass_g',
3           height = 4.5, aspect = 0.618, order = 2)
4 plt.show()
```



Trend Lines: `lmplot()`

LOWESS Smoother



The **LOWESS** (Locally weighted Scatterplot Smoothing) method fits a nonparametric regression line to show the overall trend.

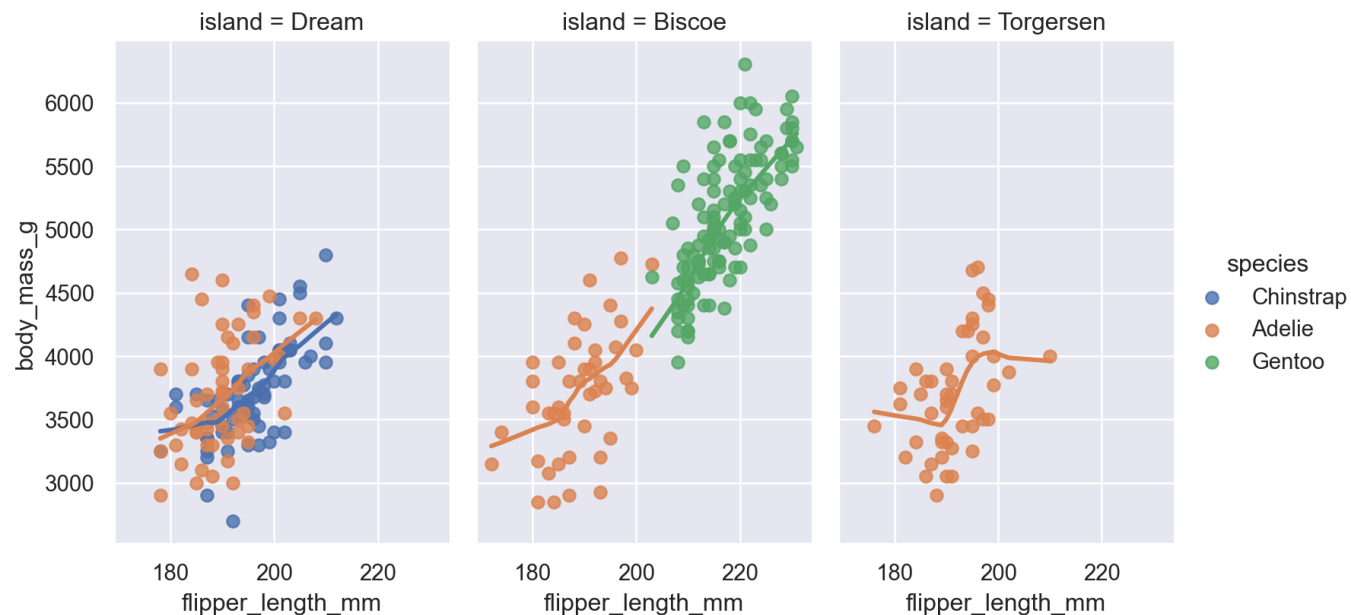
- A locally weighted linear regression around each point.
- Especially useful when data show non-linear patterns.

Trend Lines: `lmpplot()`

LOWESS Smoother

To apply the smoother, set `lowess = True`. Requires the `statsmodels` library.

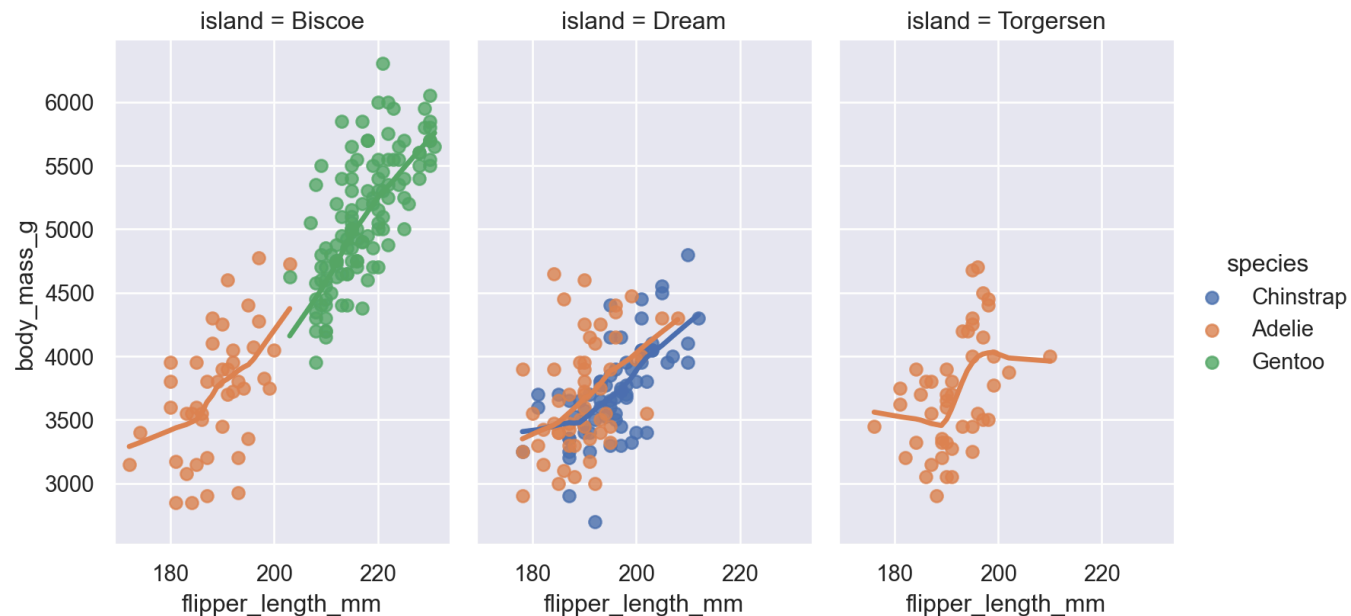
```
1 sns.lmplot(penguins, hue = 'species', col = 'island',
2           x = 'flipper_length_mm', y = 'body_mass_g',
3           height = 4.5, aspect = 0.618, lowess = True)
4 plt.show()
```



Trend Lines: `lmpplot()`

Column Order and Line Width

```
1 sns.lmplot(penguins, hue = 'species', col = 'island',
2           x = 'flipper_length_mm', y = 'body_mass_g',
3           height = 4.5, aspect = 0.618, lowess = True,
4           col_order = ['Biscoe', 'Dream', 'Torgersen'],
5           line_kws = {'linewidth': 2.5})
6 plt.show()
```



Trend Lines: `lmplot()`

Title, Labels, Other Customisations

Recall that `lmplot()` returns a `FacetGrid` object, where each subplot is a single **axes**.

- To adjust **axis** labels for each **axes**, we use the object-oriented approach.

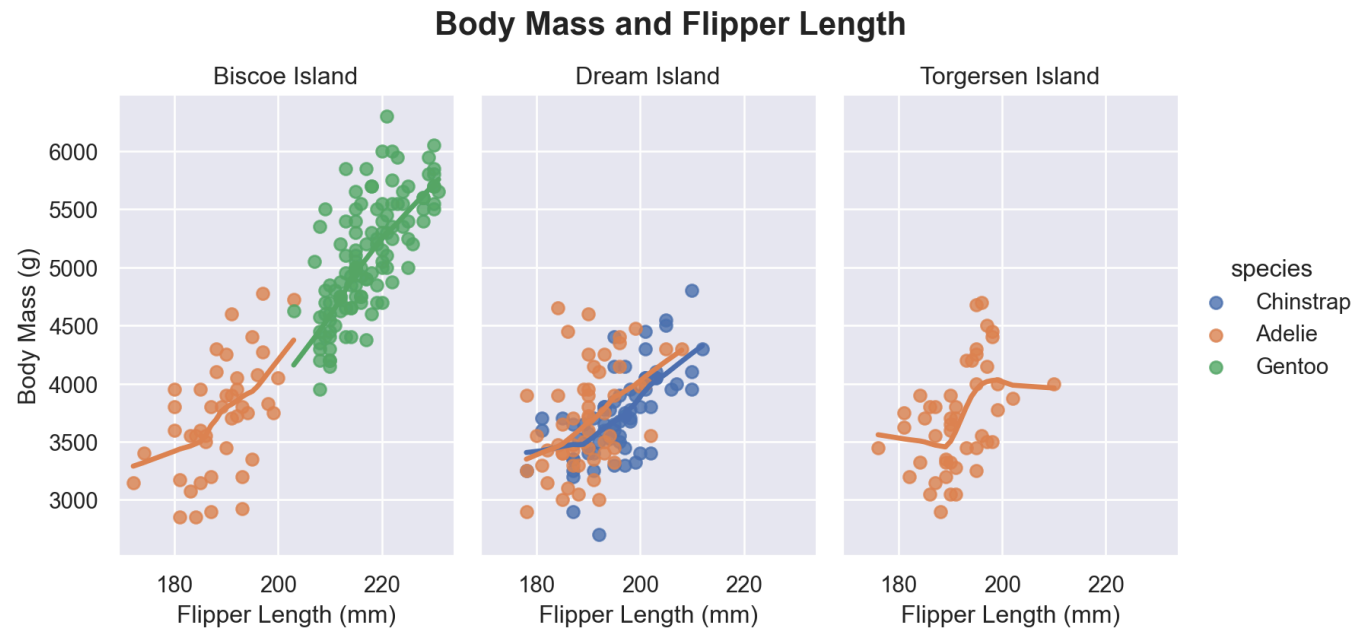
```
1 g = sns.lmplot(penguins, hue = 'species', col = 'island',
2               x = 'flipper_length_mm', y = 'body_mass_g',
3               height = 4.5, aspect = 0.618, lowess = True,
4               col_order = ['Biscoe', 'Dream', 'Torgersen'],
5               line_kws = {'linewidth': 2.5})
6 g.set_axis_labels('Flipper Length (mm)', 'Body Mass (g)')
7 g.set_titles(col_template = '{col_name} Island')
8 g.figure.suptitle('Body Mass and Flipper Length',
9                  fontsize = 16, fontweight = 'bold')
10 g.figure.subplots_adjust(top = 0.85)
11 plt.show()
```



Trend Lines: `lmpLot()`

Title, Labels, Other Customisations

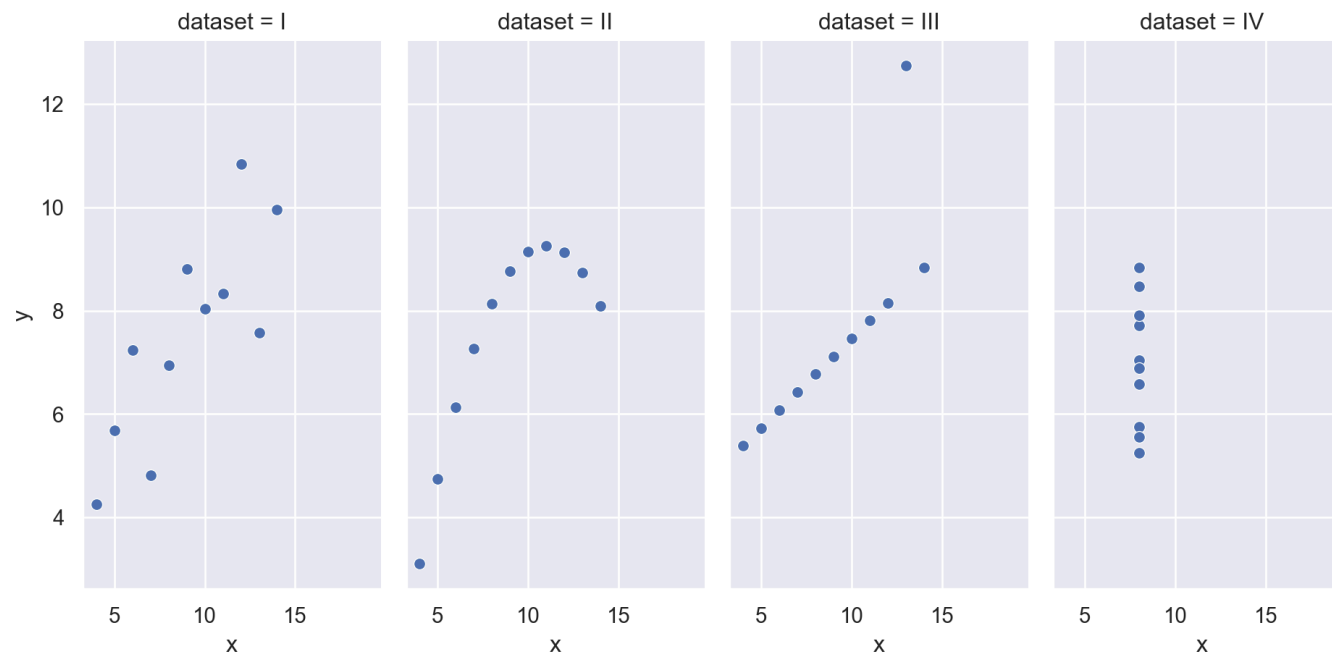
Here is how the plot panel looks like:



Case Study: Anscombe's Quartet

- In the visualisation principals lecture, we discussed the Anscombe's Quartet.
- Each data set has nearly identical mean, variance, and correlation, but different relationships.

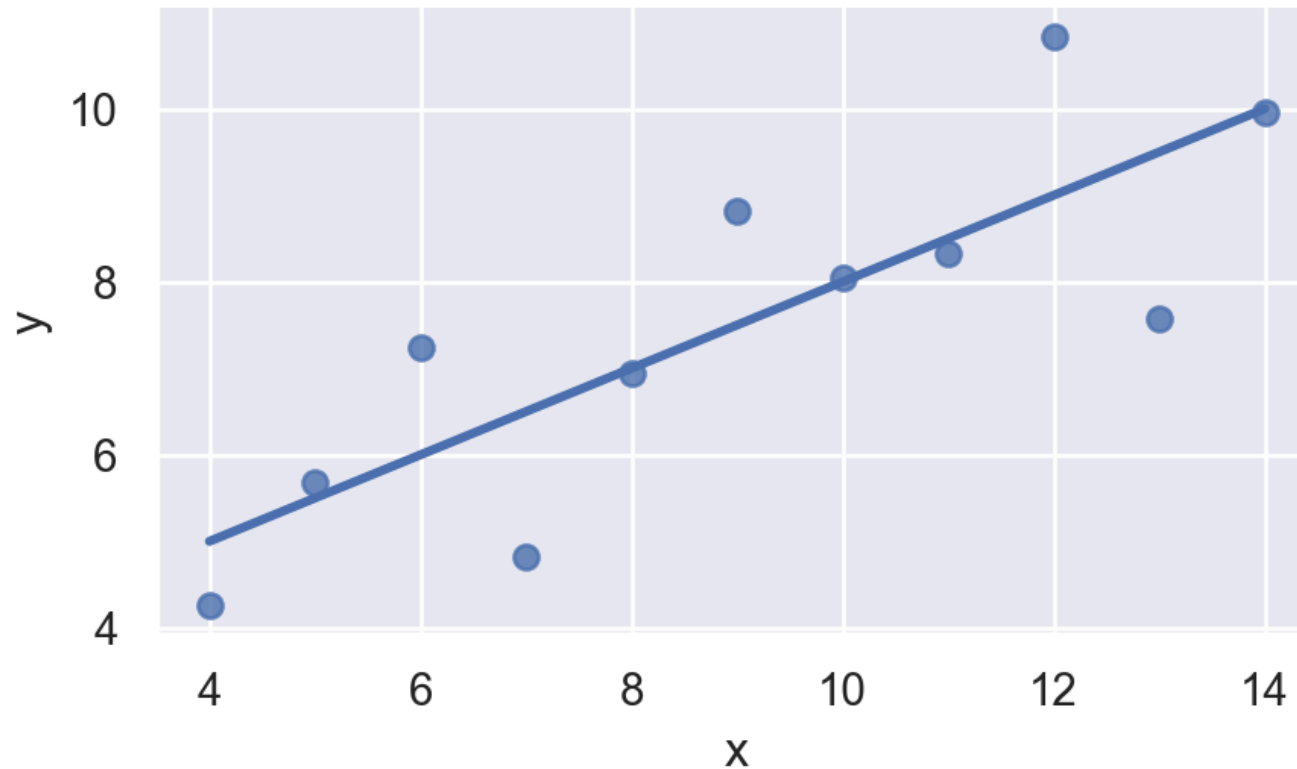
```
1 anscombe = pd.read_csv('../data/anscombe.csv')
2 sns.relplot(anscombe, x = 'x', y = 'y', col = 'dataset',
3             height = 5, aspect = 0.5)
```



Case Study: Anscombe's Quartet

Data Set I: Linear Fit

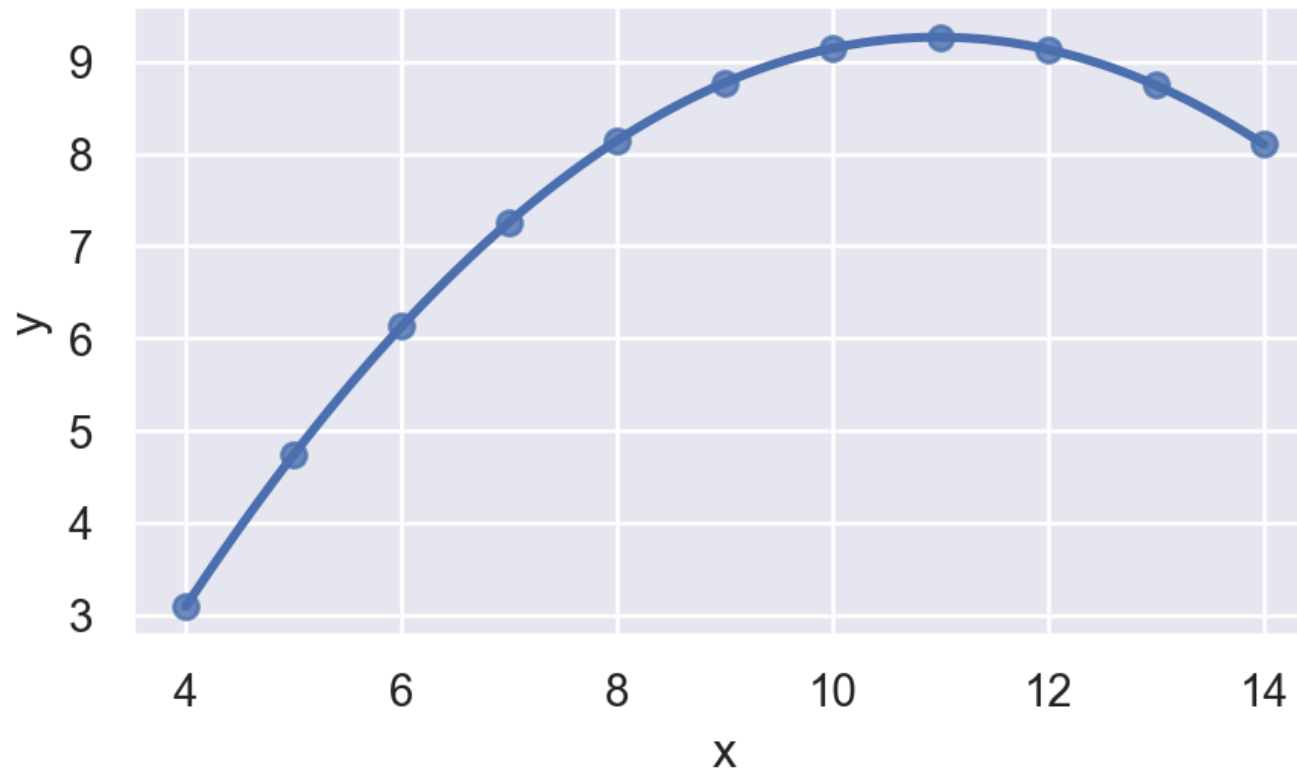
```
1 df1 = anscombe[anscombe['dataset'] == 'I']  
2 sns.lmplot(df1, x = 'x', y = 'y', ci = None,  
3           height = 3, aspect = 1.618)  
4 plt.show()
```



Case Study: Anscombe's Quartet

Data Set II: Quadratic fit

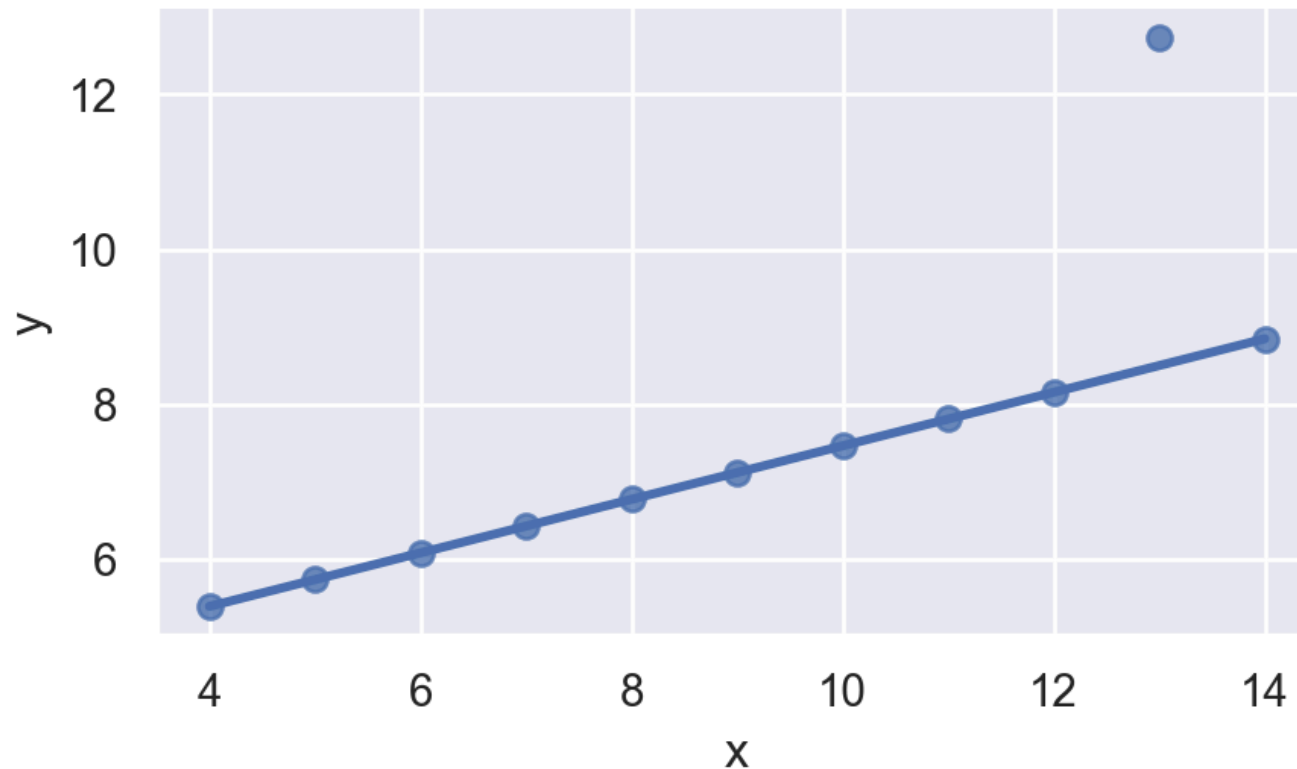
```
1 df2 = anscombe[anscombe['dataset'] == 'II']
2 sns.lmplot(df2, x = 'x', y = 'y', ci = None, order = 2,
3             height = 3, aspect = 1.618)
4 plt.show()
```



Case Study: Anscombe's Quartet

Data Set III: Robust Regression to “Downweigh” Outlier

```
1 df3 = anscombe[anscombe['dataset'] == 'III']
2 sns.lmplot(df3, x = 'x', y = 'y', ci = None, robust = True,
3           height = 3, aspect = 1.618)
4 plt.show()
```



Case Study: Nobel Prizes Data

Let us revisit the Nobel Prize data.

We shall apply what we have learned today to visualise the same data:

- Age of the winners at the time of award by field.

```
1 df = pd.read_csv('../data/wk10_nobel_prizes16.csv')
2 nobel = df[df['gender'].isin(['Female', 'Male'])]
3 nobel.sample(3, random_state = 1014)
```

	laureateID	category	fullName	year	age	gender	bornCountry	organizationCountry
14	572	Literature	Bjornstjerne Bjornson	1903	71.0	Male	Norway	NaN
91	177	Chemistry	Fritz Haber	1918	50.0	Male	Poland	Germany
309	365	Medicine	Edward Tatum	1958	55.0	Male	USA	USA

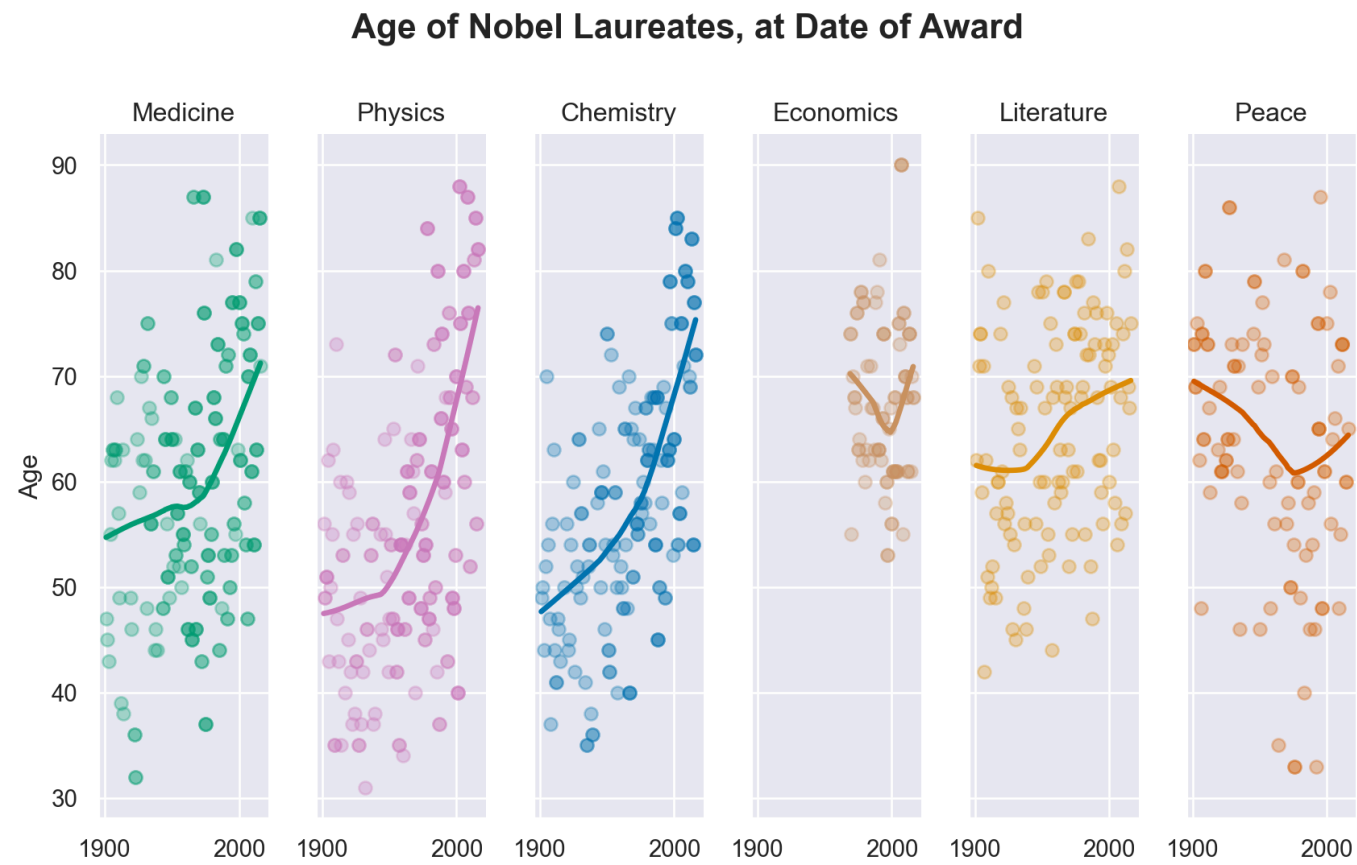
Case Study: Nobel Prizes Data

Age of Winners Across Fields

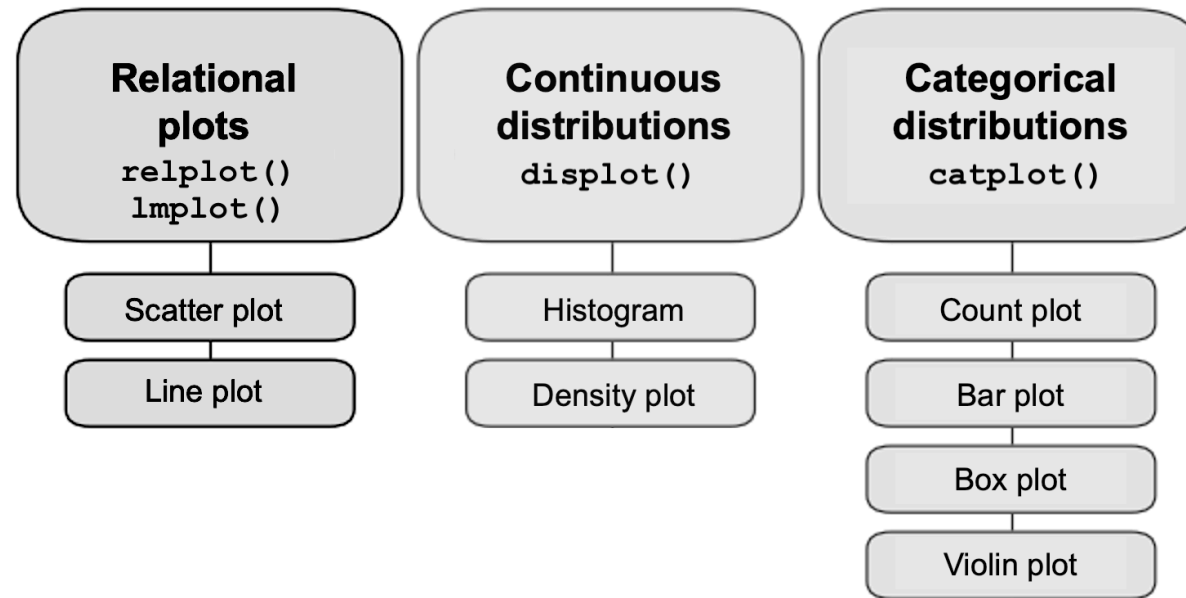
```
1 colors = sns.color_palette('colorblind', 6)
2 g = sns.lmplot(nobel, x = 'year', y = 'age',
3               hue = 'category', col = 'category',
4               col_order = ['Medicine', 'Physics', 'Chemistry',
5                           'Economics', 'Literature', 'Peace'],
6               lowess = True, height = 6, aspect = 0.25, palette = colors,
7               scatter_kws = {'alpha': 0.3}, line_kws = {'linewidth': 2.5})
8 g.set_axis_labels('', 'Age')
9 g.set_titles('{col_name}')
10 g.figure.suptitle('Age of Nobel Laureates, at Date of Award',
11                  fontsize = 16, fontweight = 'bold')
12 g.figure.subplots_adjust(top = 0.85)
13 plt.show()
```

Case Study: Nobel Prizes Data

Age of Winners Across Fields



Continuous Distributions: `displot()`



`displot()` visualises **univariate** or **bivariate distributions** of data.

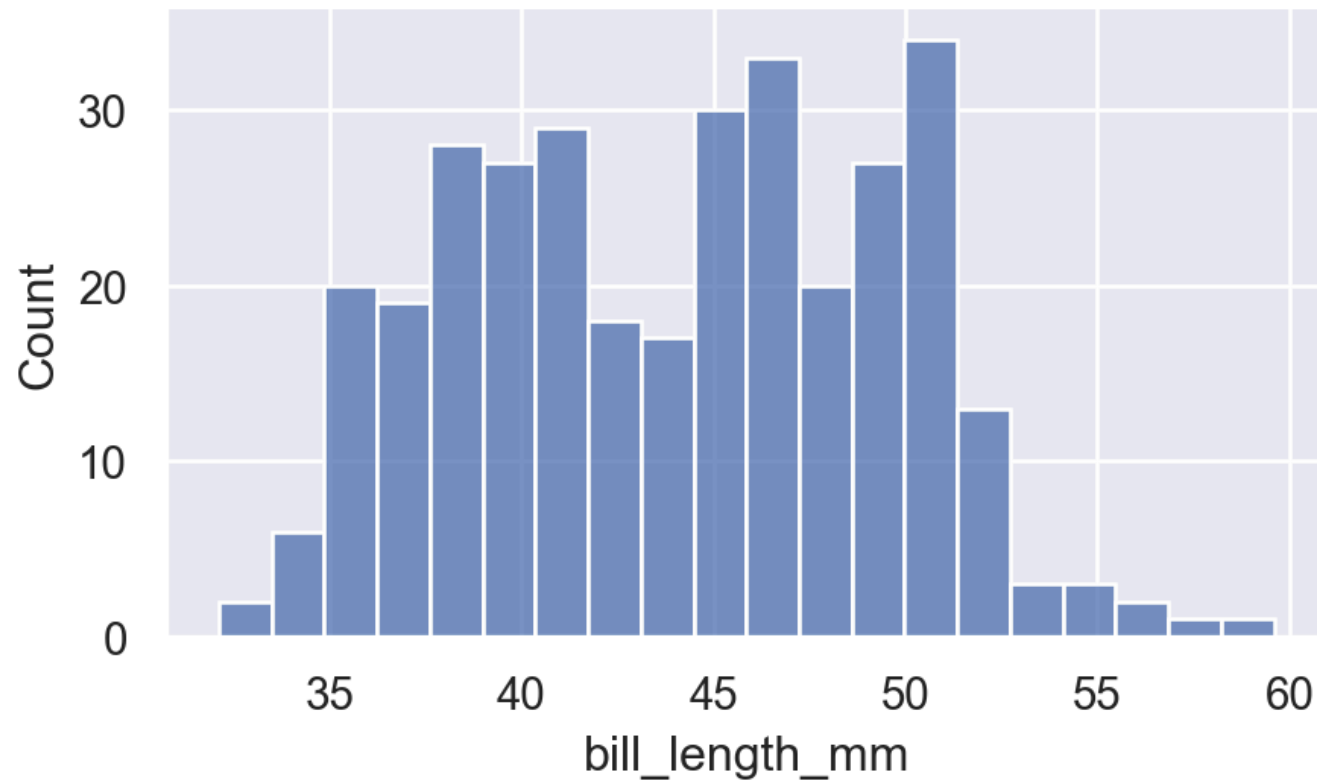
- `kind = 'hist'`: Histogram (the default).
- `kind = 'kde'`: Density plot.

Continuous Distributions: `displot()`

Histogram

Bill length of all penguins – number of bins specified.

```
1 sns.displot(penguins, x = 'bill_length_mm', kind = 'hist',  
2             height = 3, aspect = 1.618, bins = 20)  
3 plt.show()
```

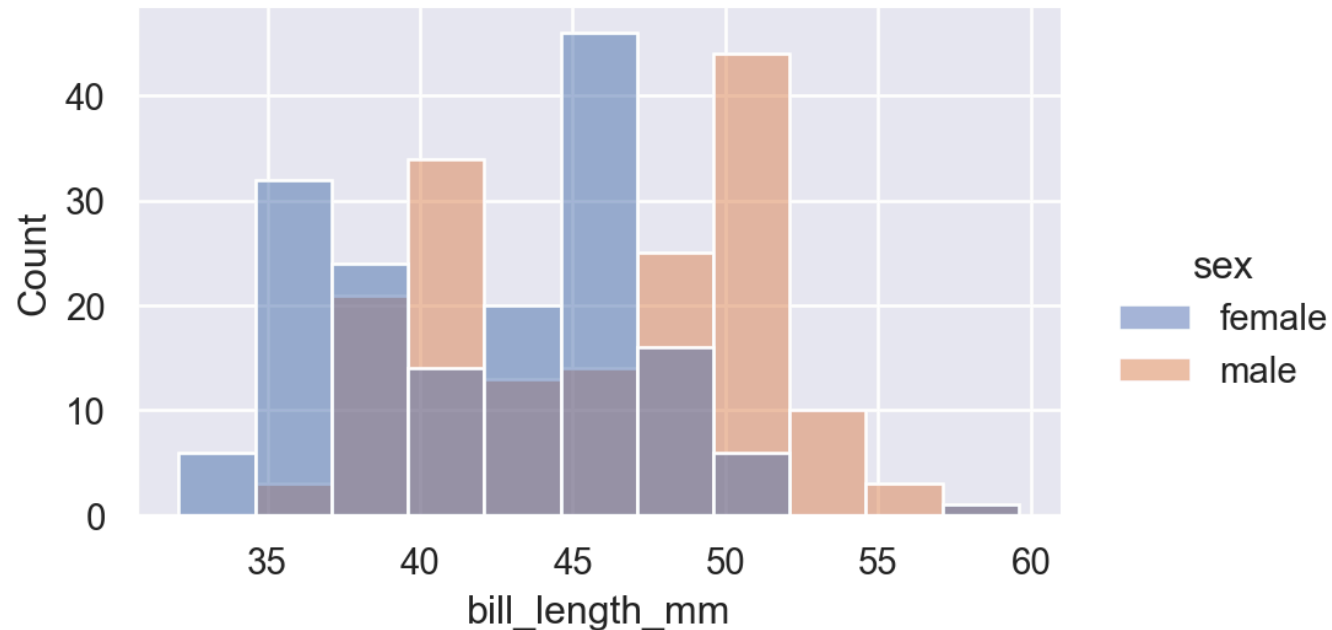


Continuous Distributions: `displot()`

Histogram: Conditioned on Categorical Variable

We can create multiple histograms using the `hue` mapping.

```
1 sns.displot(penguins, x = 'bill_length_mm',  
2             kind = 'hist', hue = 'sex',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

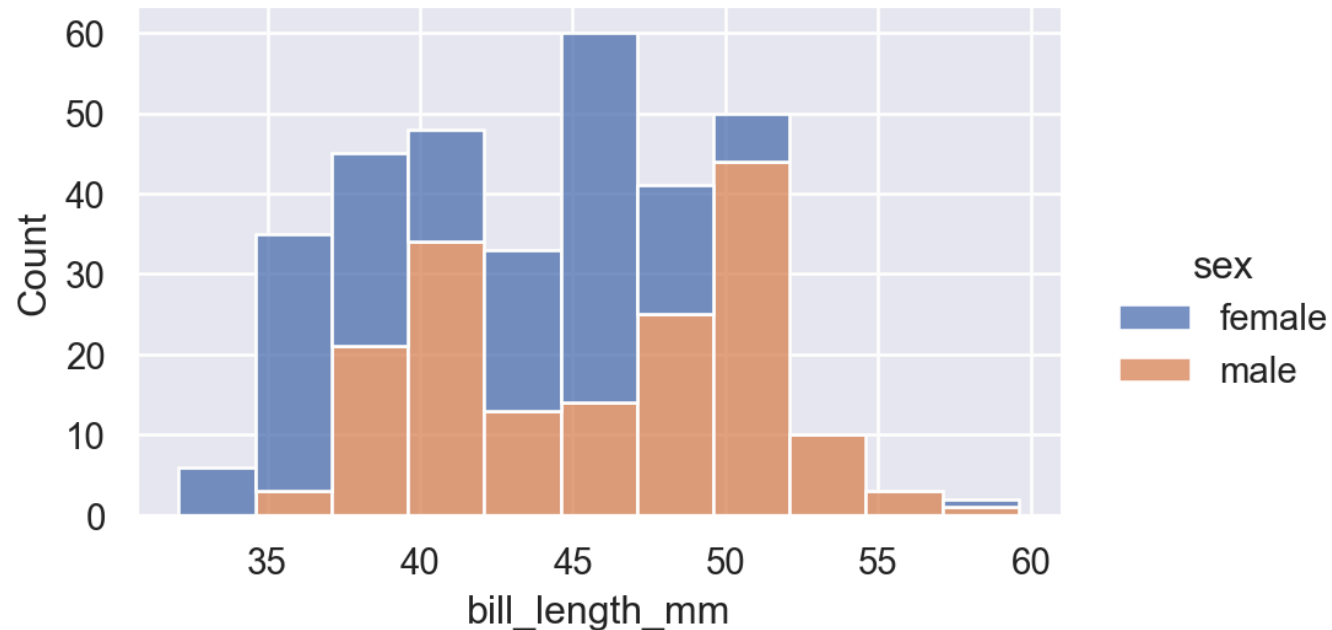


Continuous Distributions: `displot()`

Histogram: Conditioned on Categorical Variable

Instead of having **overlapping** histograms, we can **stack** them vertically.

```
1 sns.displot(penguins, x = 'bill_length_mm',  
2             kind = 'hist', hue = 'sex', multiple = 'stack',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

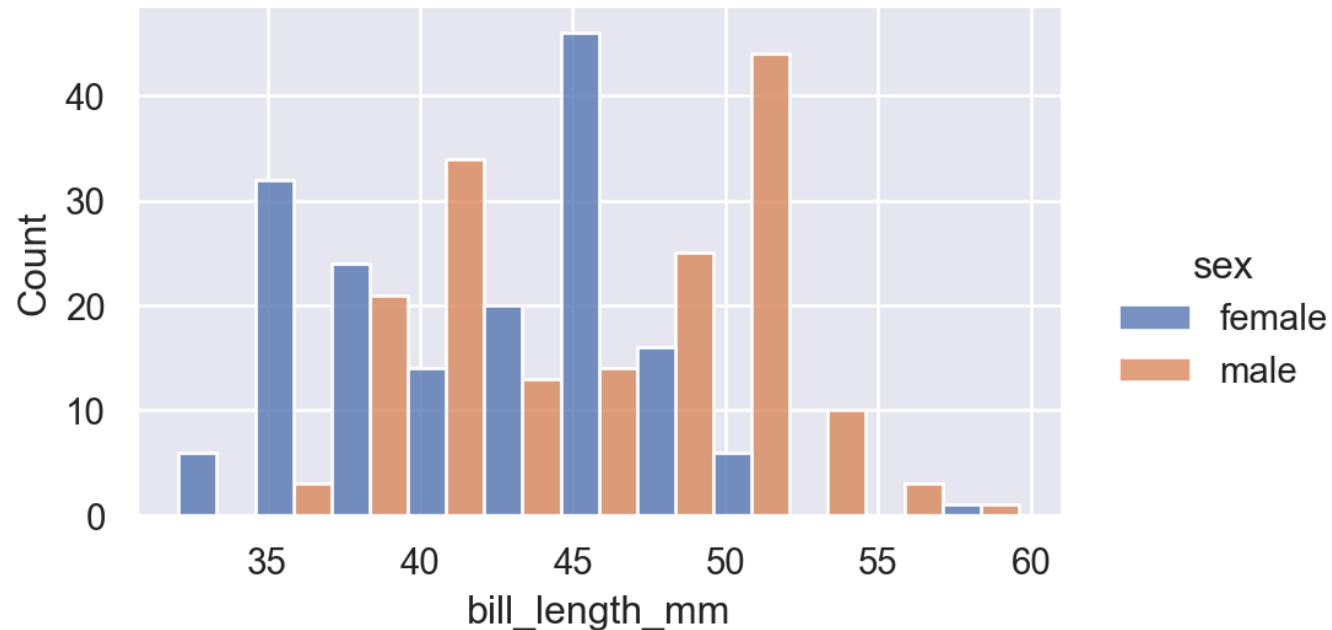


Continuous Distributions: `displot()`

Histogram: Conditioned on Categorical Variable

... or **dodge** the bars, which moves them horizontally.

```
1 sns.displot(penguins, x = 'bill_length_mm',  
2             kind = 'hist', hue = 'sex', multiple = 'dodge',  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

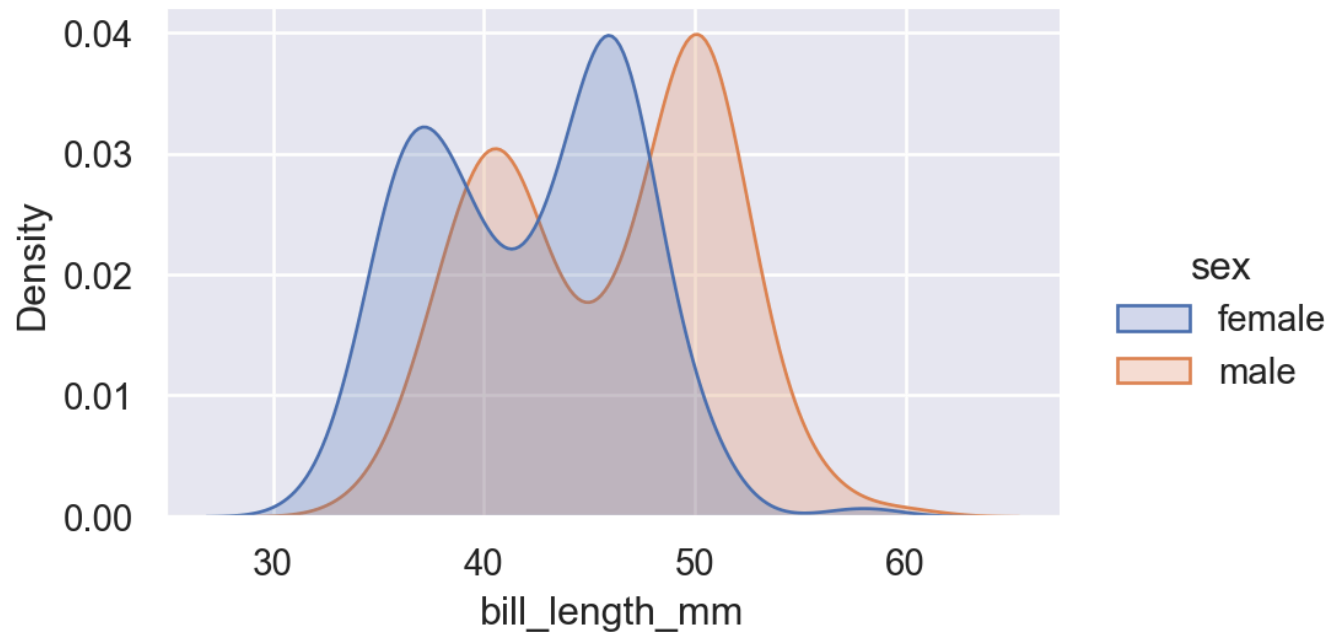


Continuous Distributions: `displot()`

Kernel Density Estimation Plot

The KDE plot provides more information about the shape of the distribution.

```
1 sns.displot(penguins, x = 'bill_length_mm',  
2             kind = 'kde', hue = 'sex', fill = True,  
3             height = 3, aspect = 1.618)  
4 plt.show()
```

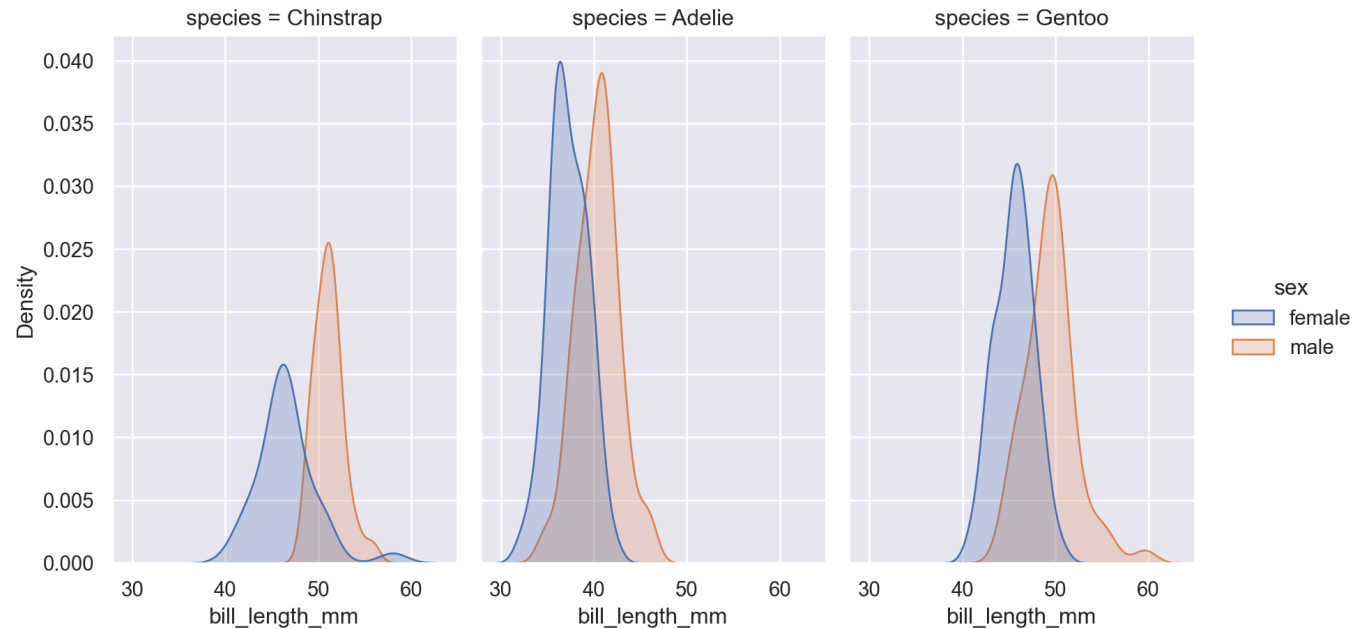


Continuous Distributions: `displot()`

Faceted Kernel Density Estimation Plot

Faceted KDE plots allow us to examine the shape across categories.

```
1 sns.displot(penguins, x = 'bill_length_mm', col = 'species',
2             kind = 'kde', hue = 'sex', fill = True,
3             height = 5, aspect = 0.618)
4 plt.show()
```



Continuous Distributions: `displot()`

Bivariate Histogram

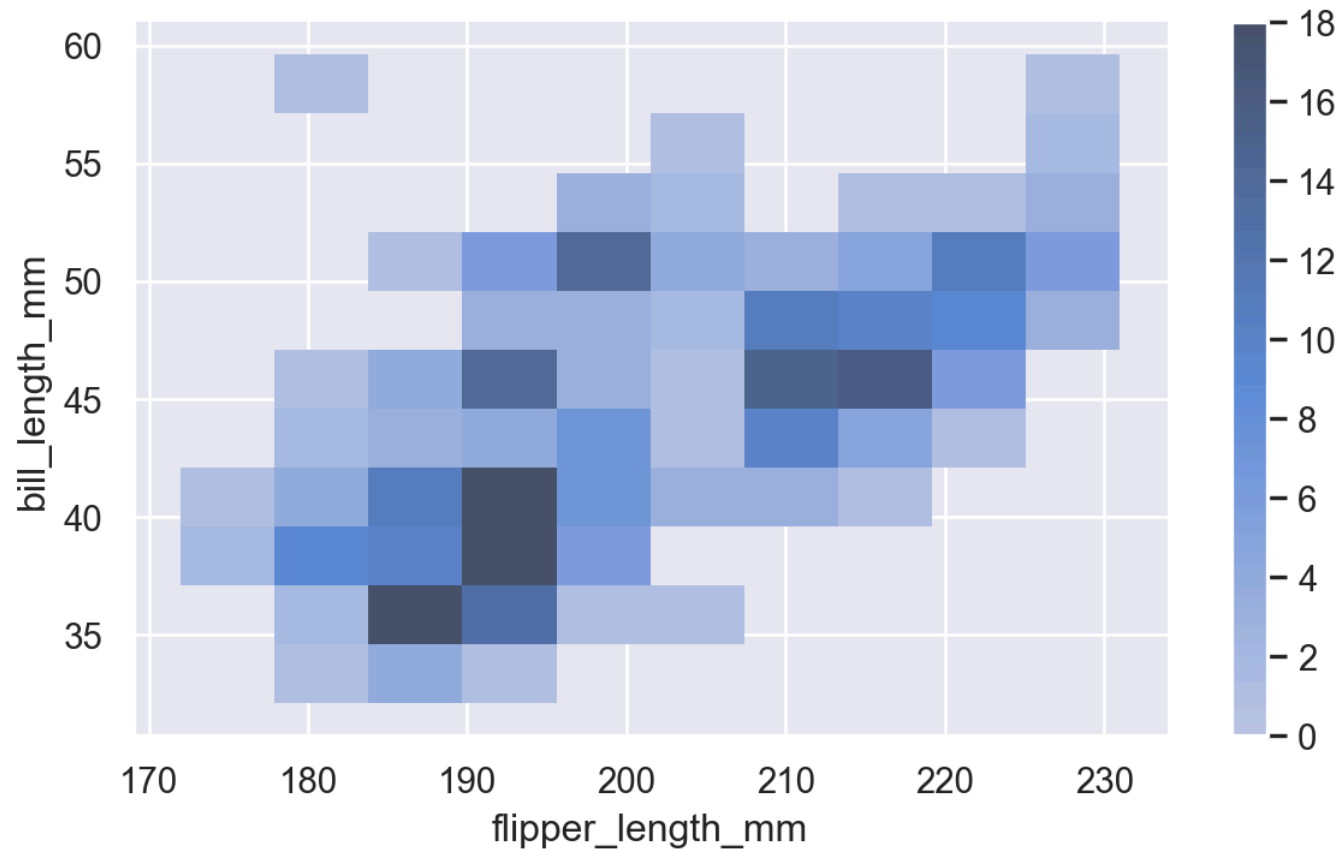
`displot()` also supports **bivariate histograms**.

- Divides the x-y plane into rectangles.
- The counts of cases falling in each rectangle are mapped to colours.
- Serves as an alternative to the scatter plot in the event of overplotting.

Continuous Distributions: `displot()`

Bivariate Histogram

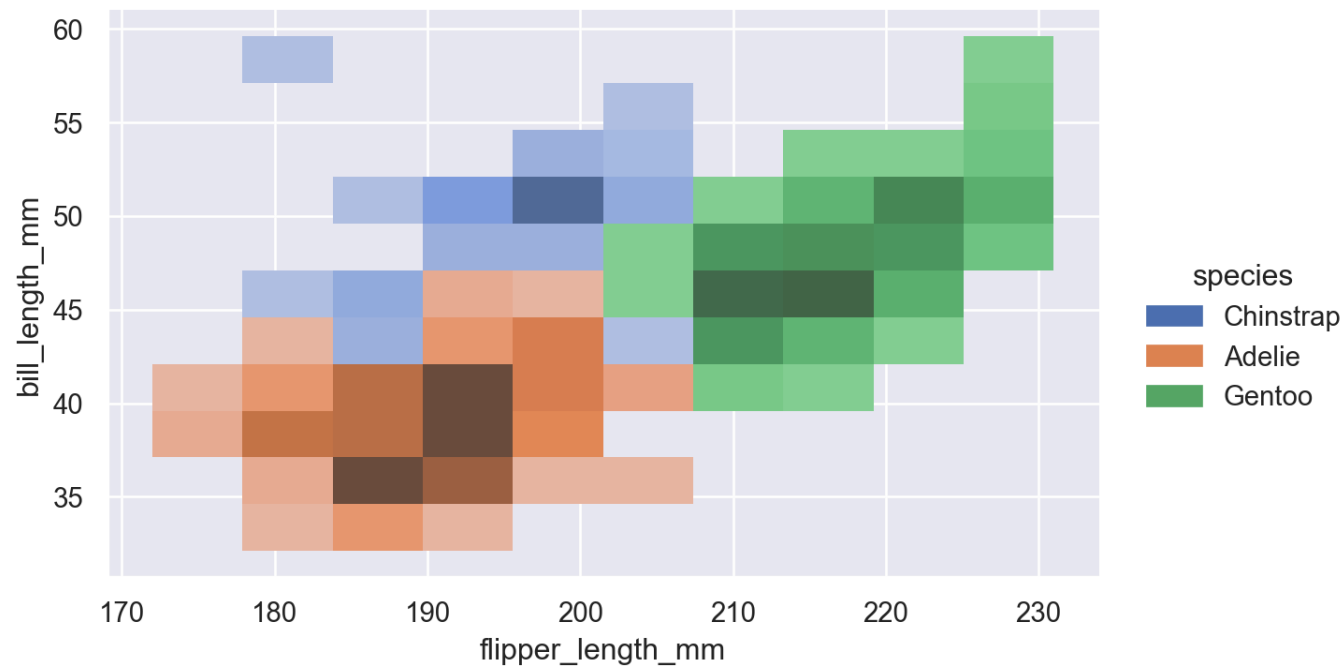
```
1 sns.displot(penguins, kind = 'hist', x = 'flipper_length_mm', y = 'bill_length_mm',  
2             height = 4, aspect = 1.618, cbar = True)  
3 plt.show()
```



Continuous Distributions: `displot()`

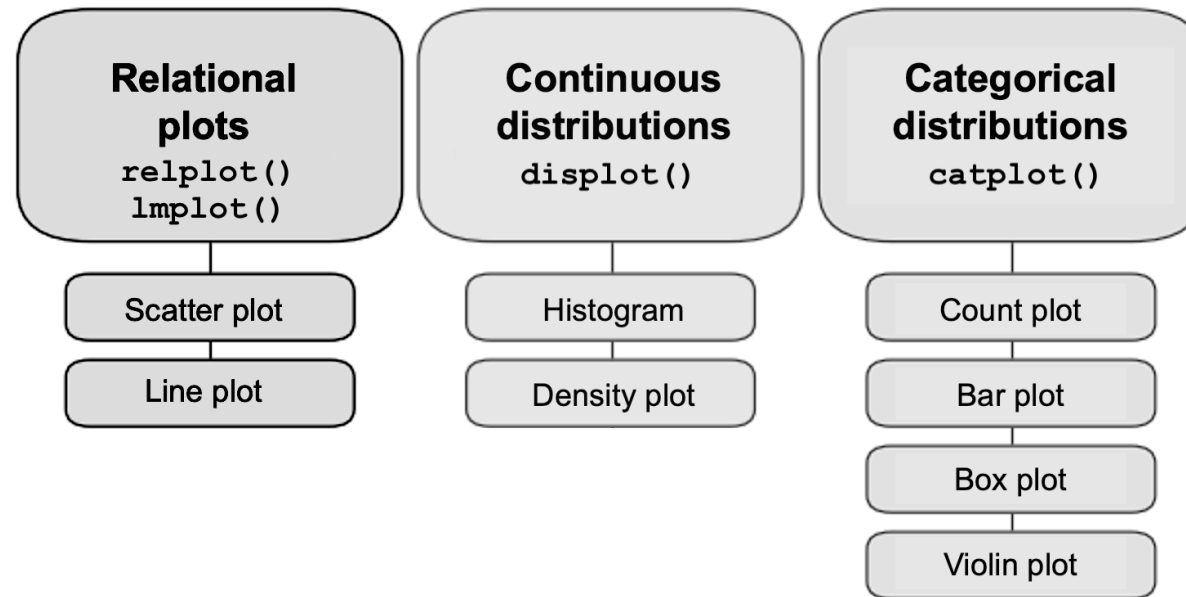
Bivariate Histogram: Conditioned on Categorical Variable

```
1 sns.displot(penguins, kind = 'hist', x = 'flipper_length_mm', y = 'bill_length_mm',
2             height = 4, aspect = 1.618, hue = 'species')
3 plt.show()
```



Additionally, we can assign an additional variable to `hue`.

Categorical Distributions: `catplot()`



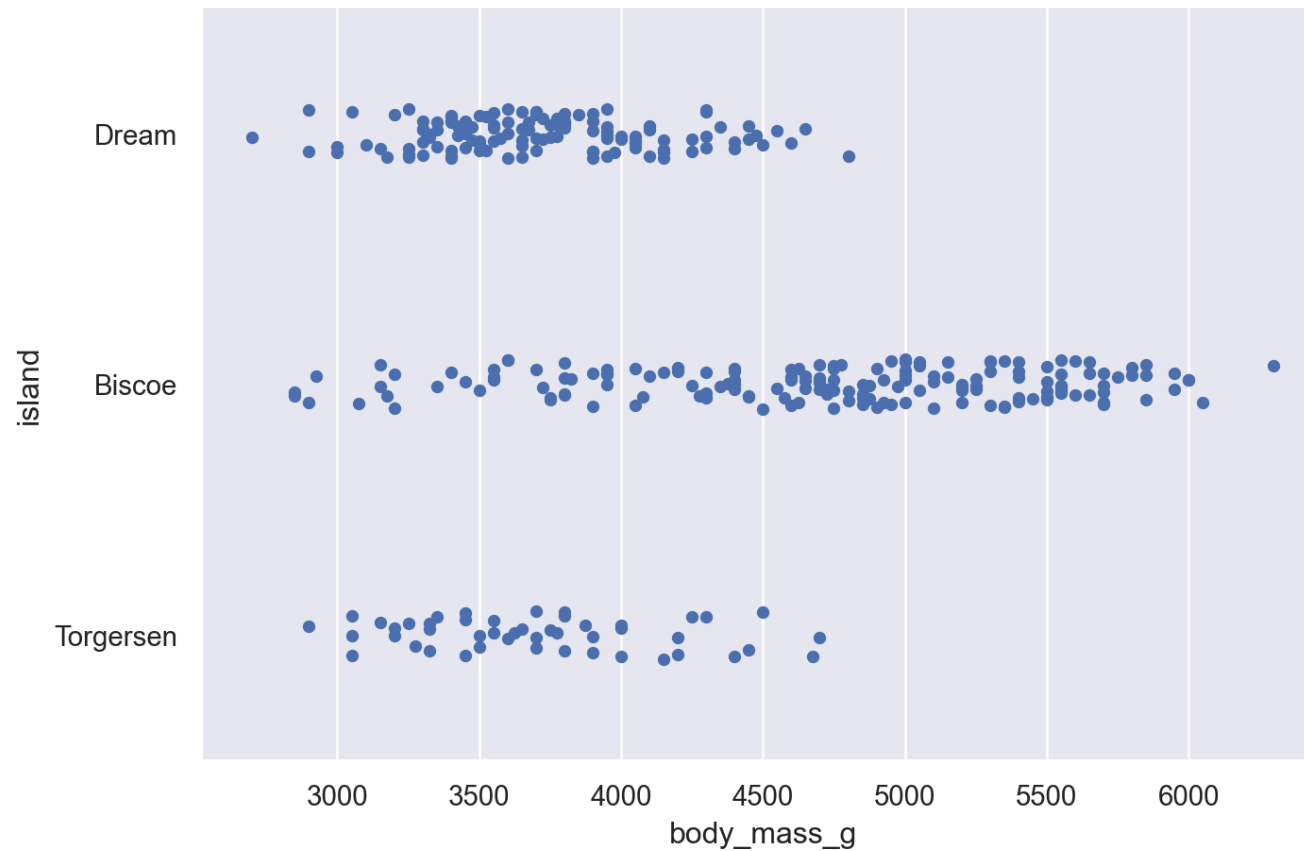
`catplot()` visualises the relationship between continuous and categorical data.

- `kind = 'strip'`: Dot plot (the default).
- `kind = 'count'`: Count plot.
- `kind = 'box'`: Box plot.
- `kind = 'violin'`: Violin plot.

Categorical Distributions: catplot()

Dot Plot: Created by default

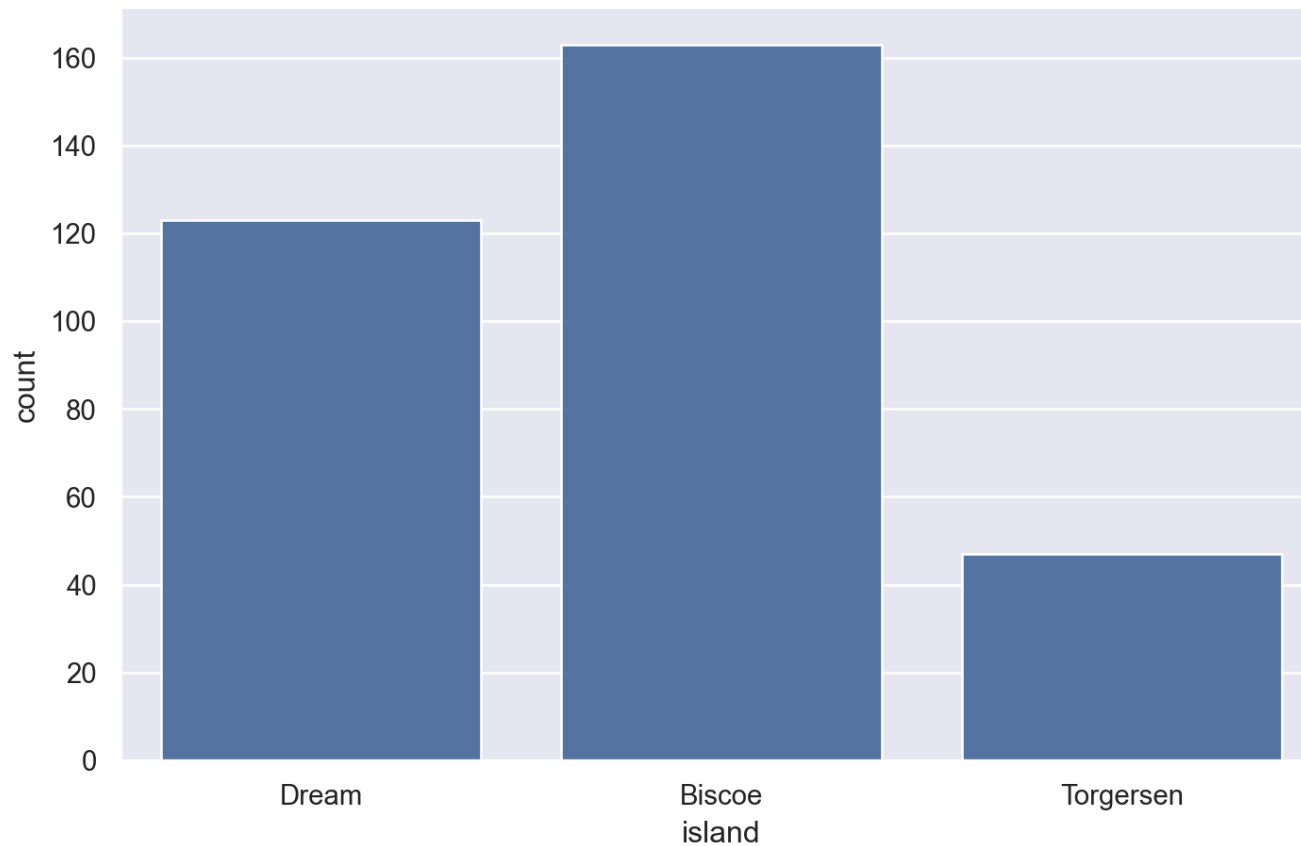
```
1 sns.catplot(penguins, x = 'body_mass_g', y = 'island',  
2             height = 5, aspect = 1.5)  
3 plt.show()
```



Categorical Distributions: catplot()

Bar Plot: Created via `kind = 'count'`

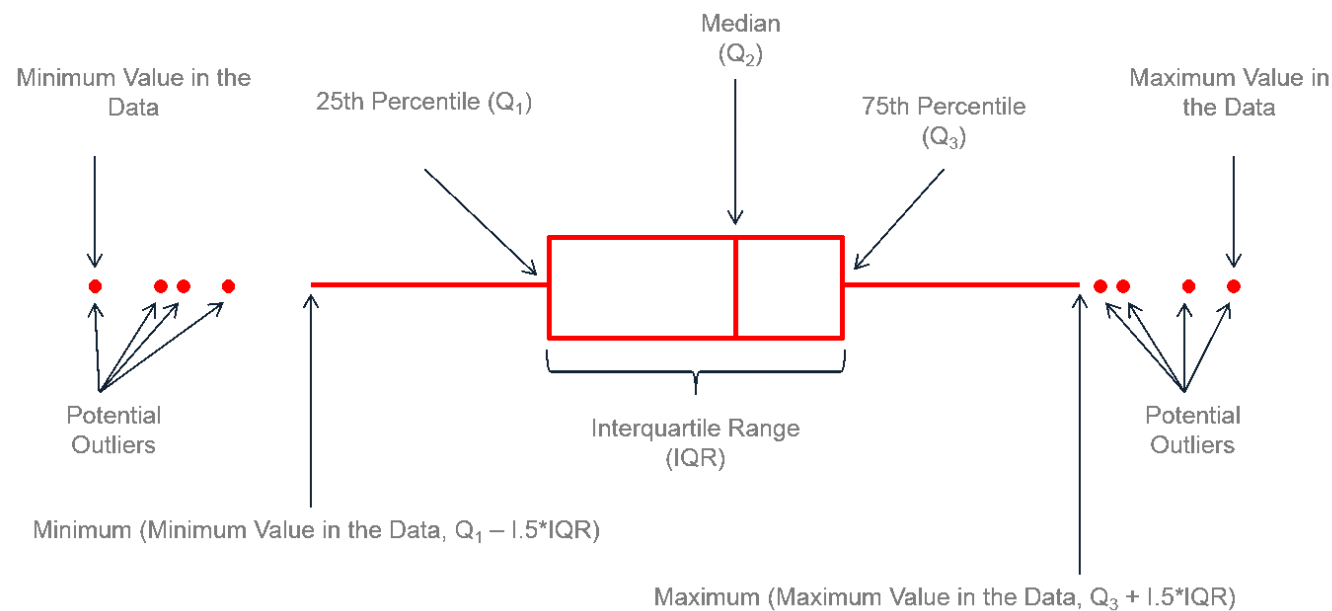
```
1 sns.catplot(penguins, x = 'island', kind = 'count',  
2             height = 5, aspect = 1.5)  
3 plt.show()
```



The Box Plot

The **Box plot** is a standard way of displaying the distribution of a continuous variable.

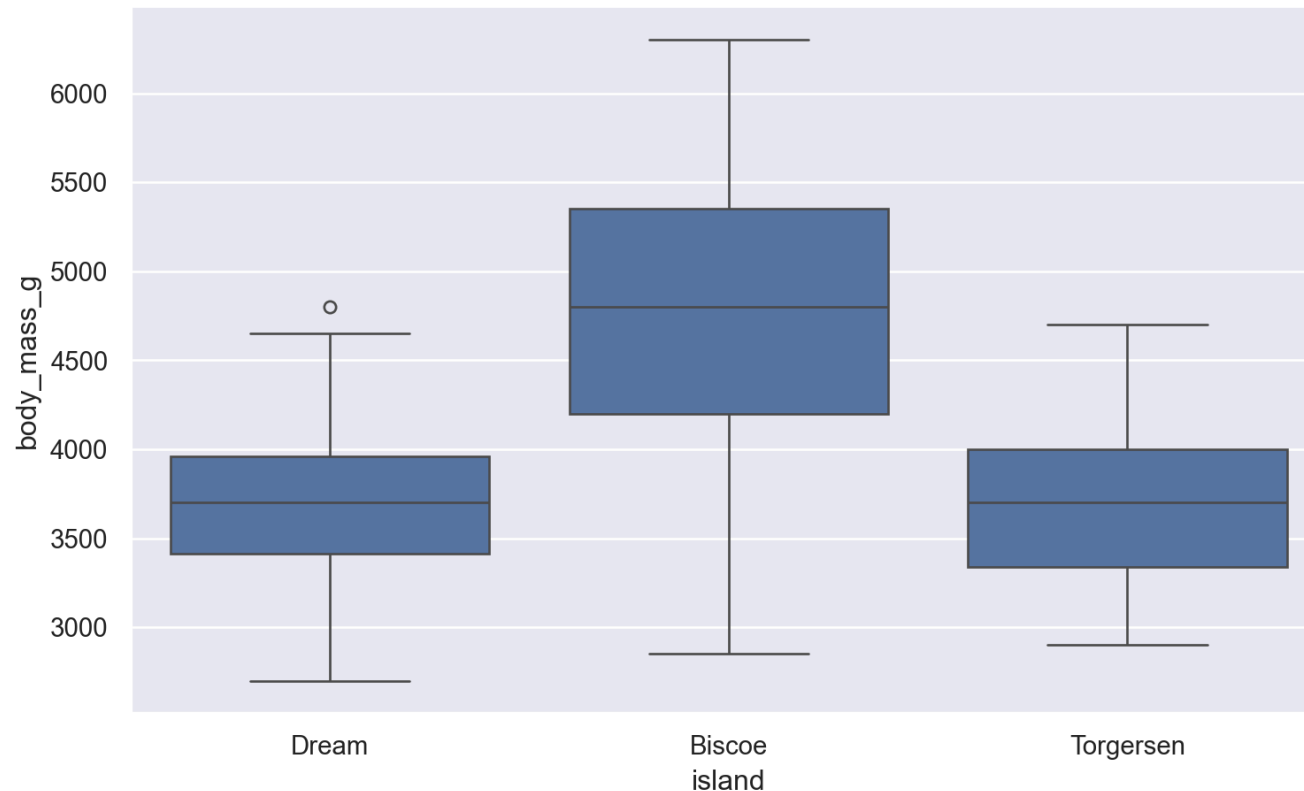
- Introduced and popularized by American statistician John Tukey.
- Especially helpful when we need to compare distributions across groups.



Categorical Distributions: catplot()

Box Plot: The following creates box plots for `body_mass_g` across `island`.

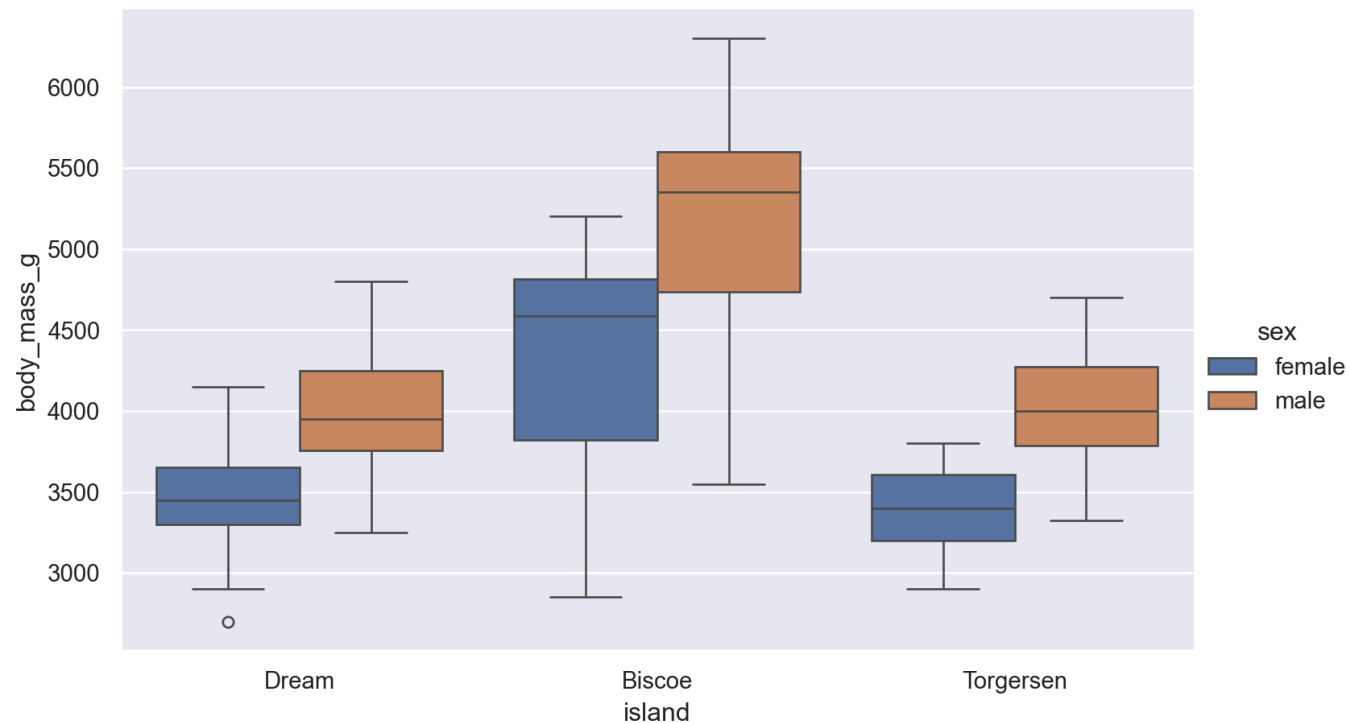
```
1 sns.catplot(penguins, kind = 'box',  
2             x = 'island', y = 'body_mass_g', height = 5,  
3             aspect = 1.6)  
4 plt.show()
```



Categorical Distributions: catplot()

Box Plot: We can also visualise `body_mass_g` across `island` and `sex`:

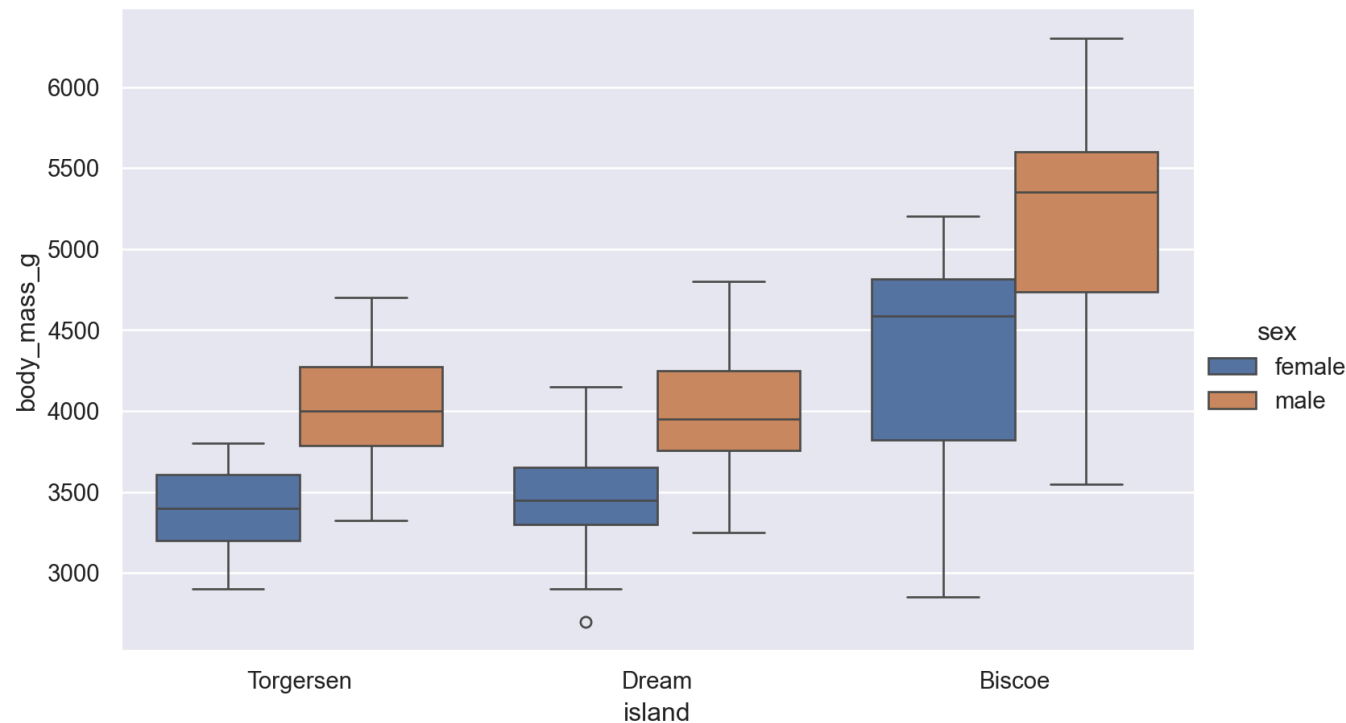
```
1 sns.catplot(penguins, kind = 'box', hue = 'sex',  
2             x = 'island', y = 'body_mass_g', height = 5,  
3             aspect = 1.6)  
4 plt.show()
```



Categorical Distributions: `catplot()`

Box Plot: We use `order` to specify the order of the categories via a list.

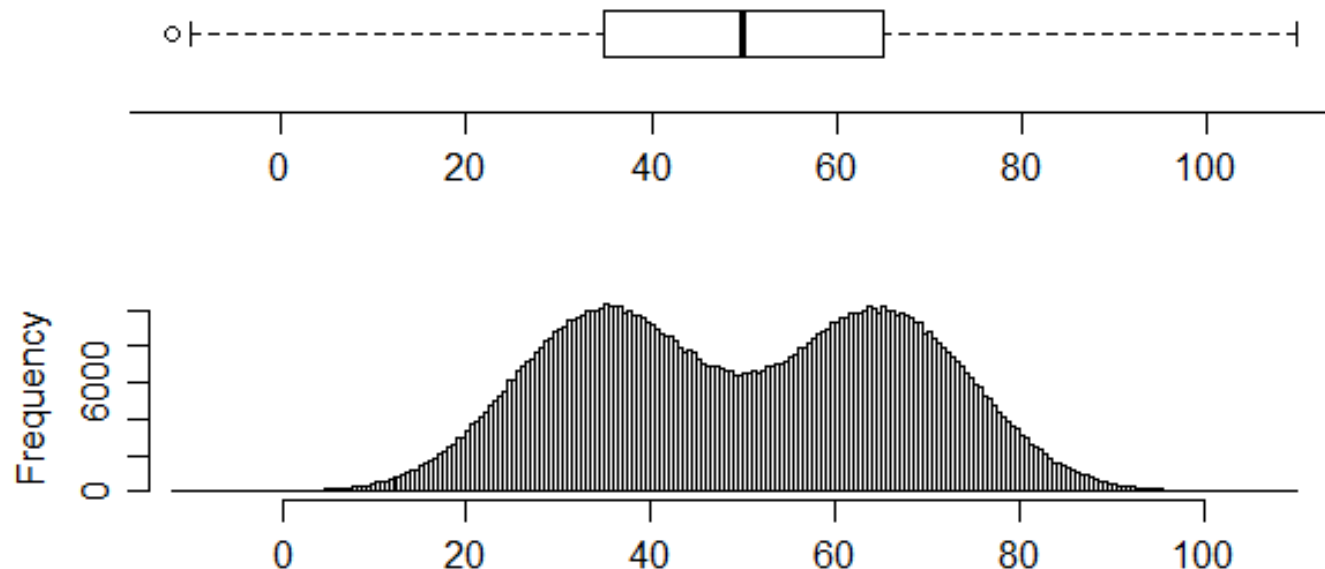
```
1 sns.catplot(penguins, kind = 'box', hue = 'sex',
2             x = 'island', y = 'body_mass_g', height = 5,
3             aspect = 1.6, order = ["Torgersen", "Dream", "Biscoe"])
4 plt.show()
```



Box Plot vs Density Plot

The box plot cannot display certain features of a distribution.

- If a distribution is **unimodal**, the box plot provides good information about the distribution.
- If a distribution is **multi-modal**, the box plot cannot tell us where the peaks are.

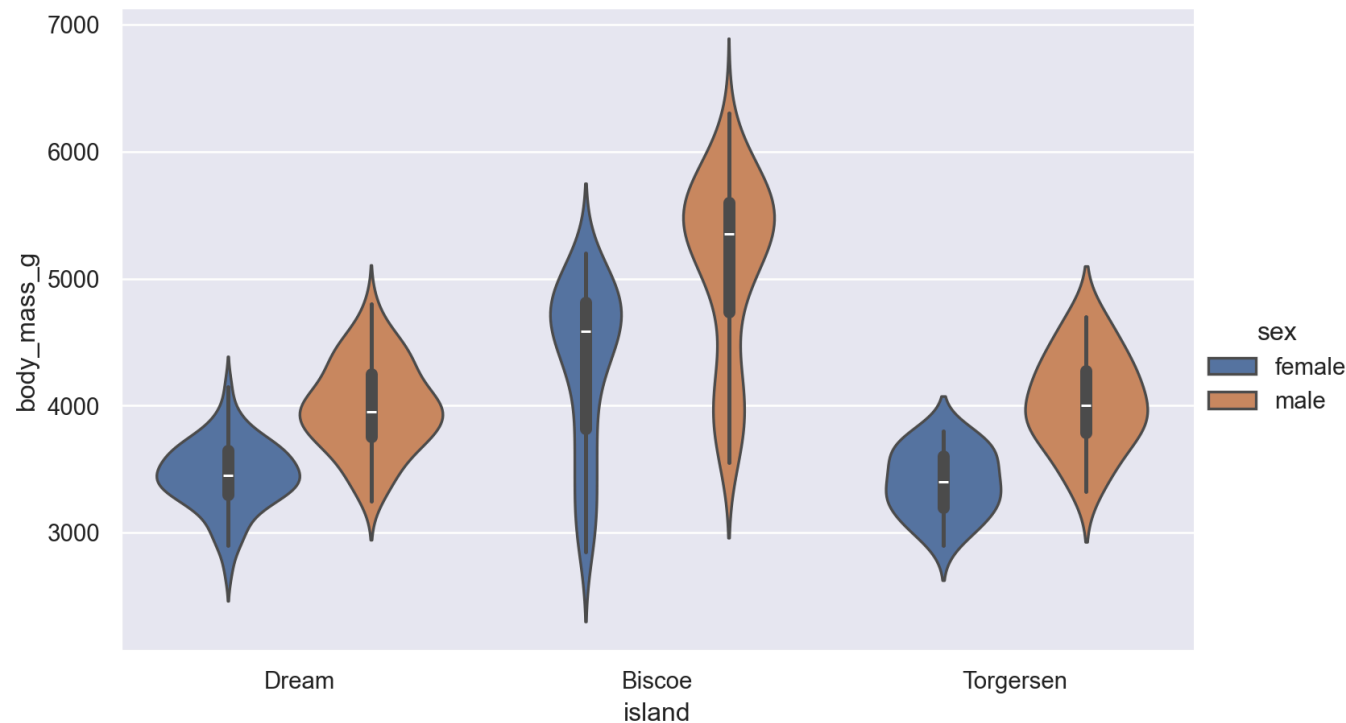


Categorical Distributions: catplot()

Violin Plot

We can use a **violin plot**. It is a hybrid of the box plot and the density plot.

```
1 sns.catplot(penguins, kind = 'violin', hue = 'sex',  
2             x = 'island', y = 'body_mass_g',  
3             height = 5, aspect = 1.6)  
4 plt.show()
```

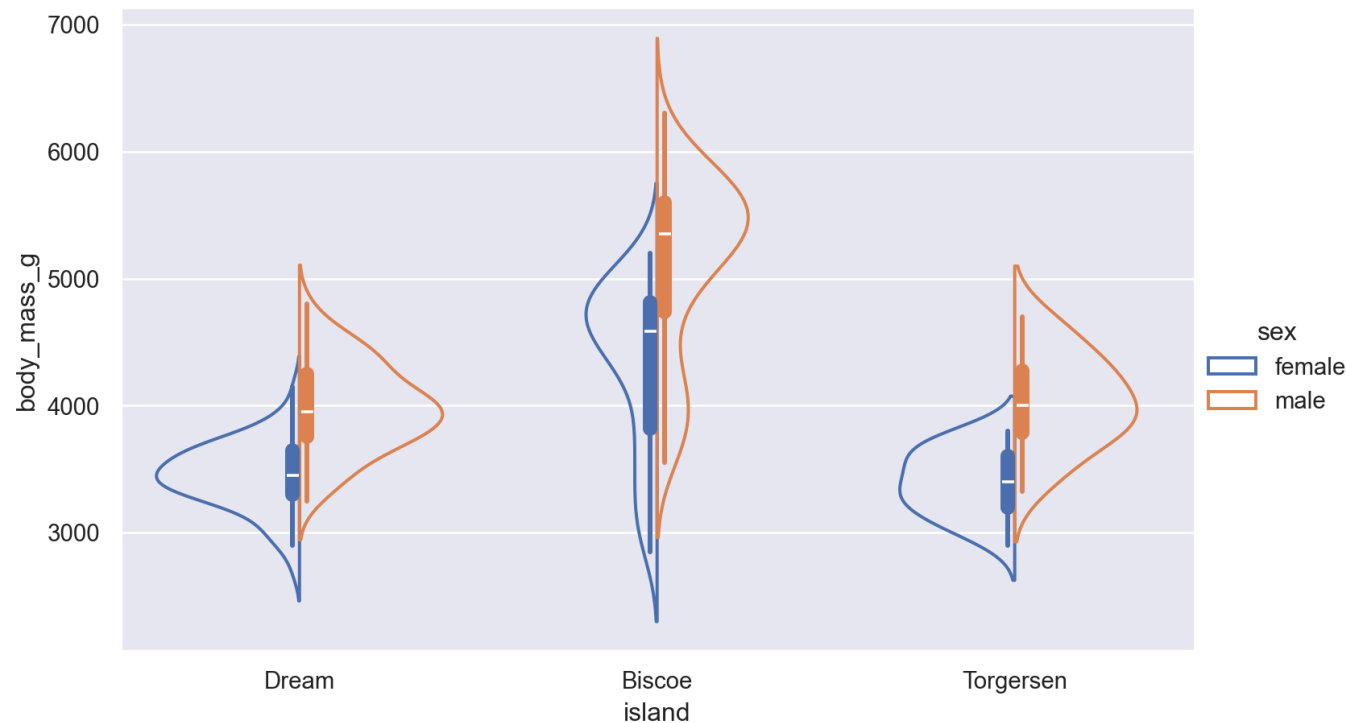


Categorical Distributions: catplot()

Violin Plot

Half of the violin is actually redundant. In fact, we can fine-tune it further.

```
1 sns.catplot(penguins, kind = 'violin', hue = 'sex',  
2             x = 'island', y = 'body_mass_g',  
3             height = 5, aspect = 1.6, split = True, fill = False)  
4 plt.show()
```

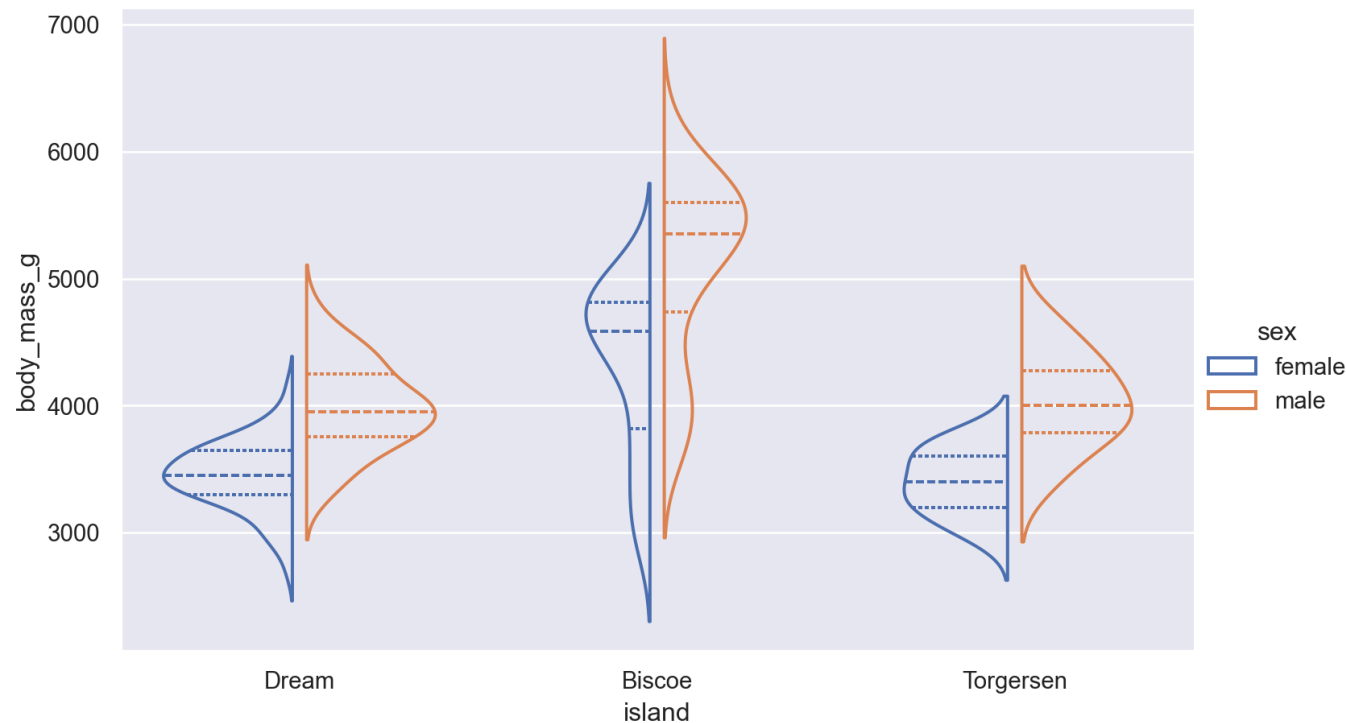


Categorical Distributions: catplot()

Violin Plot

Use `inner = 'quart'` to show quartiles and `gap` to dodge the violins.

```
1 sns.catplot(penguins, kind = 'violin', hue = 'sex',  
2             x = 'island', y = 'body_mass_g', inner = "quart", gap = 0.1,  
3             height = 5, aspect = 1.6, split = True, fill = False)  
4 plt.show()
```



Plotting Many Distributions: `pairplot()`

The `pairplot()` function uses a **small multiple** approach to visualise distributions.

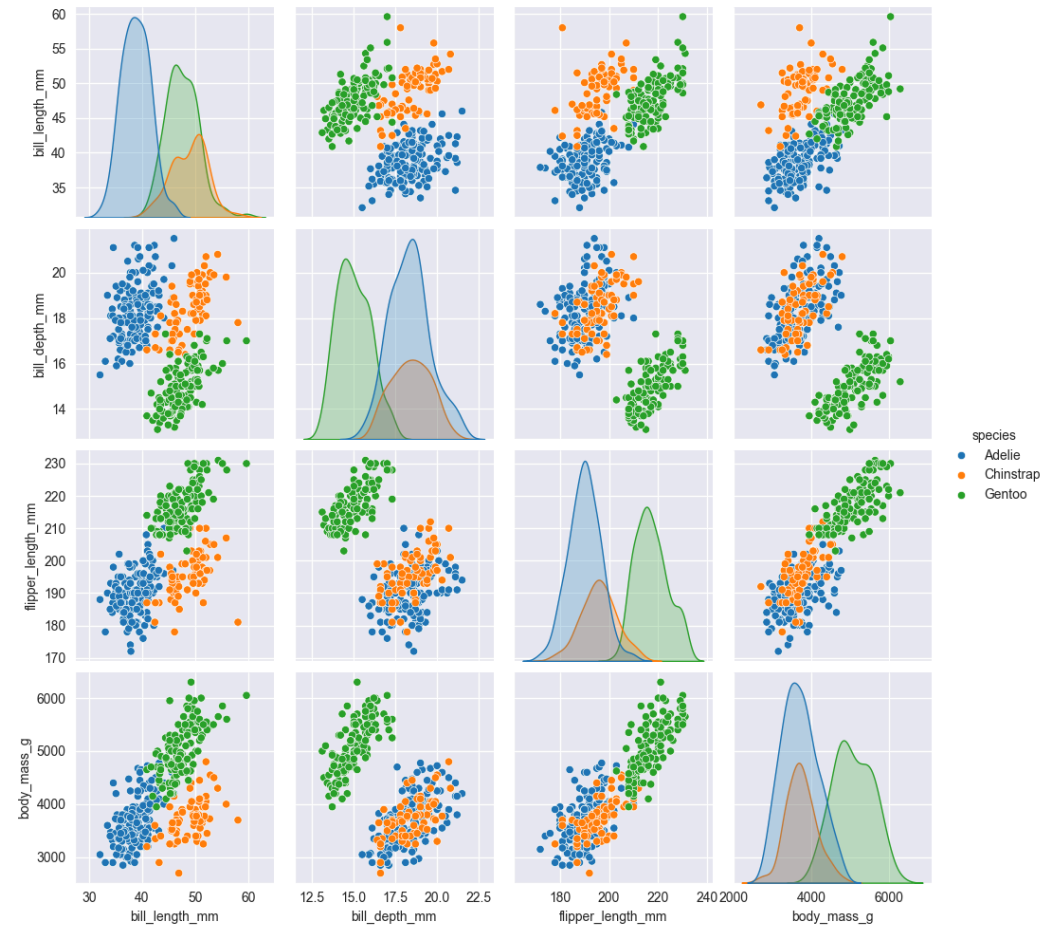
- Univariate distribution (histograms) and bivariate distribution (scatter plots).
- By default, it creates a grid of **axes** and visualises pair-wise relationships between all numeric variables.

```
1 sns.pairplot(penguins, hue = 'species')  
2 plt.show()
```



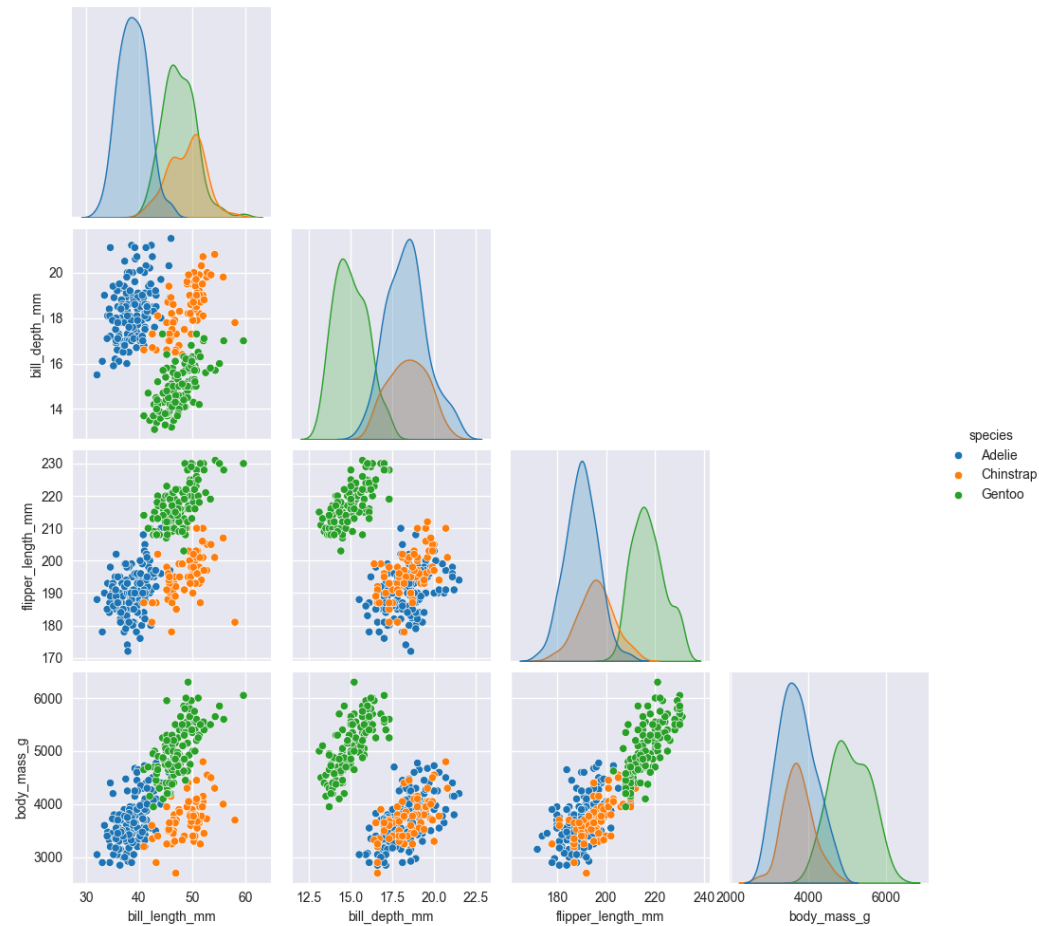
Plotting Many Distributions: pairplot()

```
1 sns.pairplot(penguins, hue = 'species')  
2 plt.show()
```



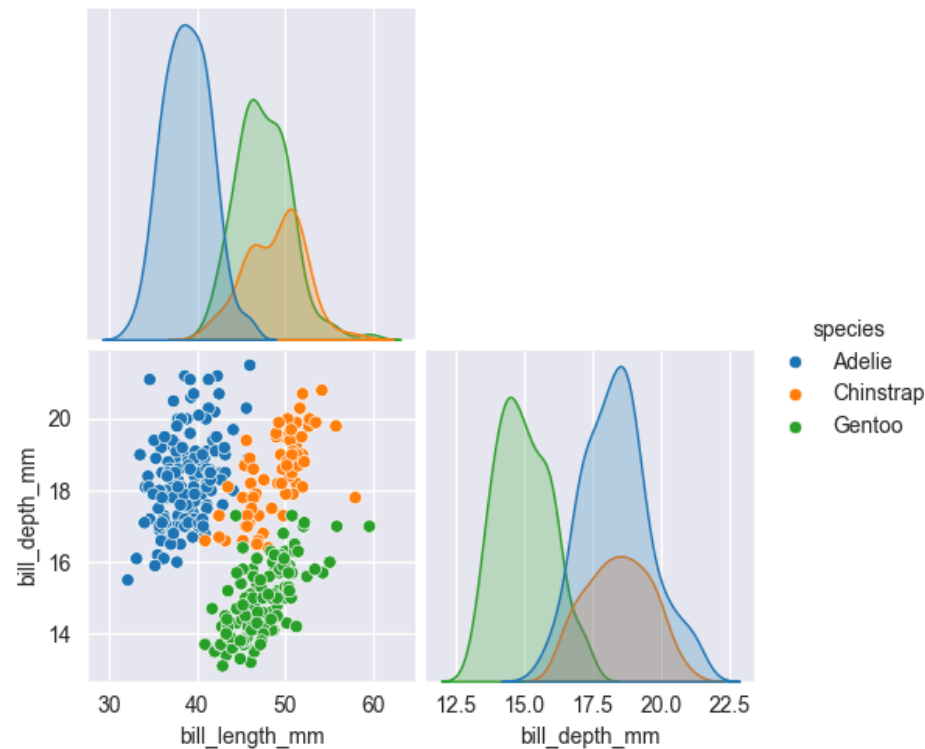
Plotting Many Distributions: pairplot()

```
1 sns.pairplot(penguins, hue = 'species', corner = True)  
2 plt.show()
```



Plotting Many Distributions: `pairplot()`

```
1 sns.pairplot(penguins, hue = 'species', corner = True,  
2             vars = ['bill_length_mm', 'bill_depth_mm'])  
3 plt.show()
```

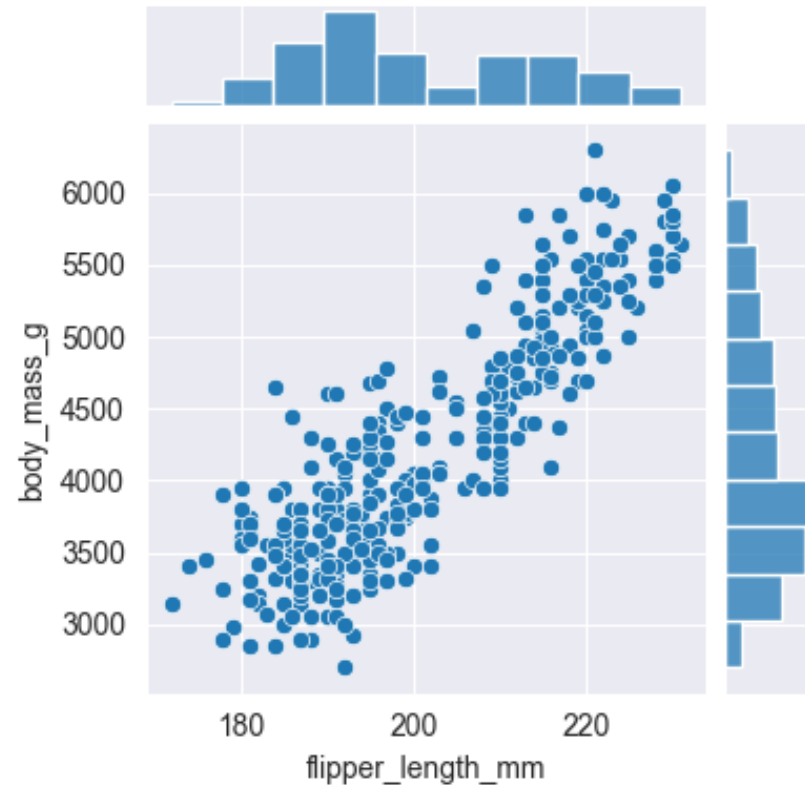


Applying `pairplot()` on a subset of of numerical variables...

Joint and Marginal Distributions: `jointplot()`

`jointplot()` plots the **joint** and **marginal distributions** of two variables.

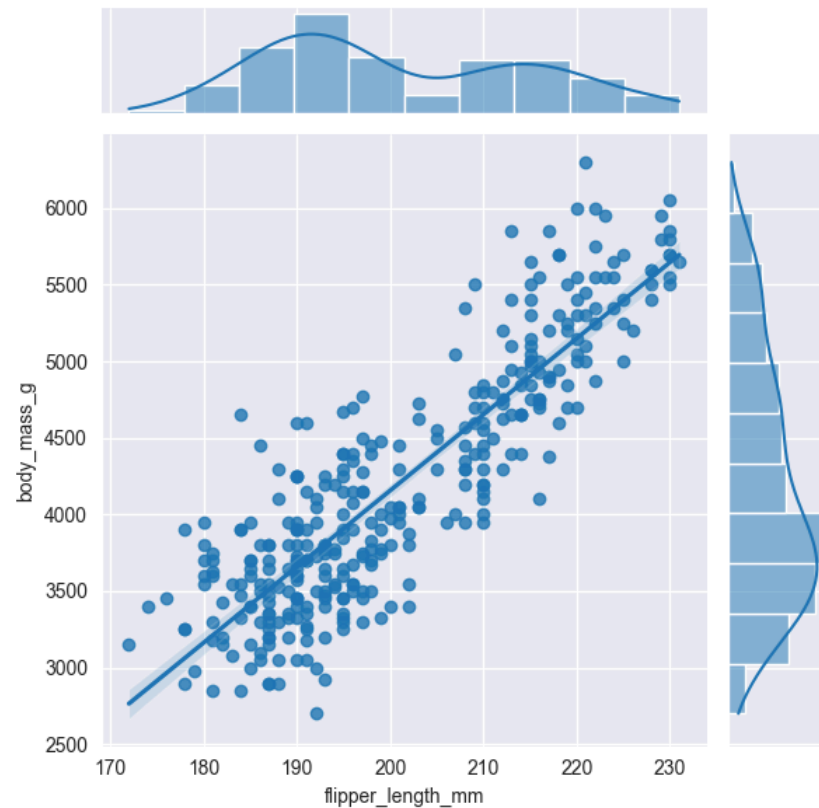
```
1 sns.jointplot(penguins,  
2               x = 'flipper_length_mm', y = 'body_mass_g')  
3 plt.show()
```



Joint and Marginal Distributions: `jointplot()`

Add a regression line and density plots via `kind = 'reg'` ...

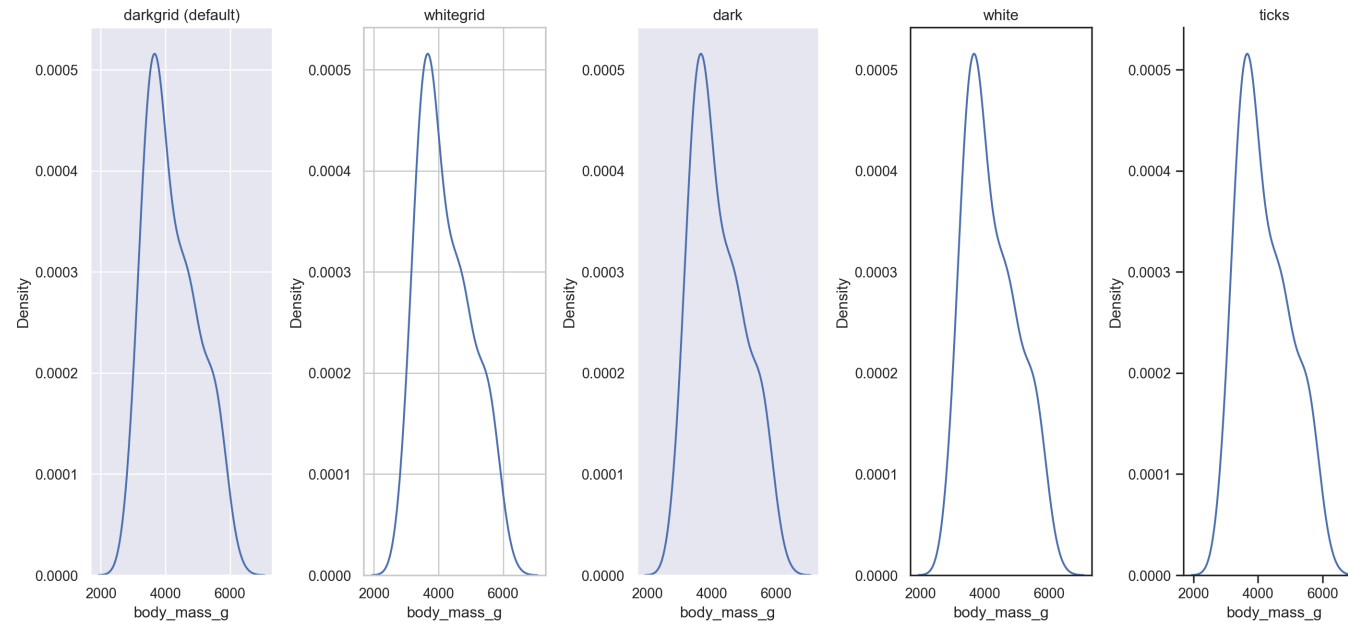
```
1 sns.jointplot(penguins, kind = 'reg',  
2               x = 'flipper_length_mm', y = 'body_mass_g')  
3 plt.show()
```



seaborn Customisation

seaborn Customisation: Themes

- There are five pre-set `seaborn` themes:



- In this lecture, we used the `darkgrid` style.

```
1 sns.set_theme(style = 'darkgrid')
```



seaborn Customisation: Edge and Interior Fill

- We can specify colors to objects on a plot, and distinguish between the color of the **edge** and the **interior fill** of the object.

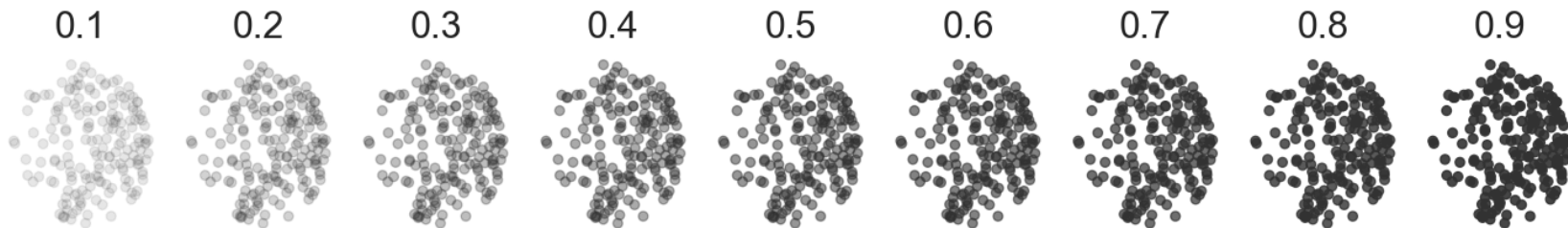


- `seaborn` automatically choose a color palette for us base on the type of the variable.



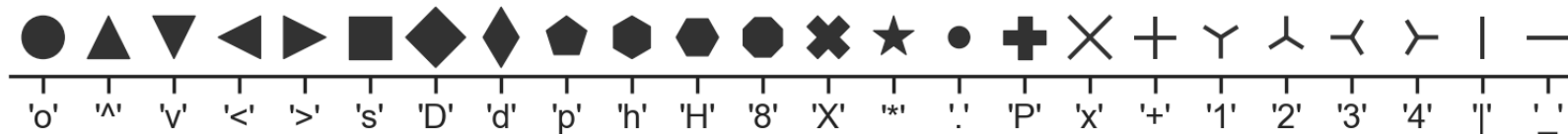
seaborn Customisation: Opacity

- The `alpha` argument take values from 0 to 1, and determines **opacity**.
- Useful when the markers on a scatter plot overlaps with each other.
- Also, colors with lower opacity appears less saturated. This can improve the appearance of the visualisation.

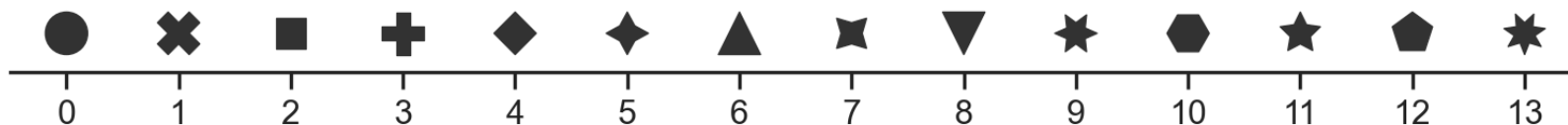


seaborn Customisation: Marker styles

- Marker styles can be specified using the following string codes:



- When a categorical is supplied to `style`, each category will be mapped with a nominal scale:
- Keep in mind though – we should still prioritize using color, instead of style, to distinguish categories in our plots.



Summary

This week we covered the basics of `seaborn`:

- **Axes-Level Functions**
- **Figure-Level Functions**
- Customisation